C3103

# Computer Networks



**Dr. Pradeepthi K V**

C R Rao Advanced Institute of Mathematics, Statistics and Computer Science

University of Hyderabad Campus, Gachibowli,

Hyderabad.

# UNIT - III

## Network Layer

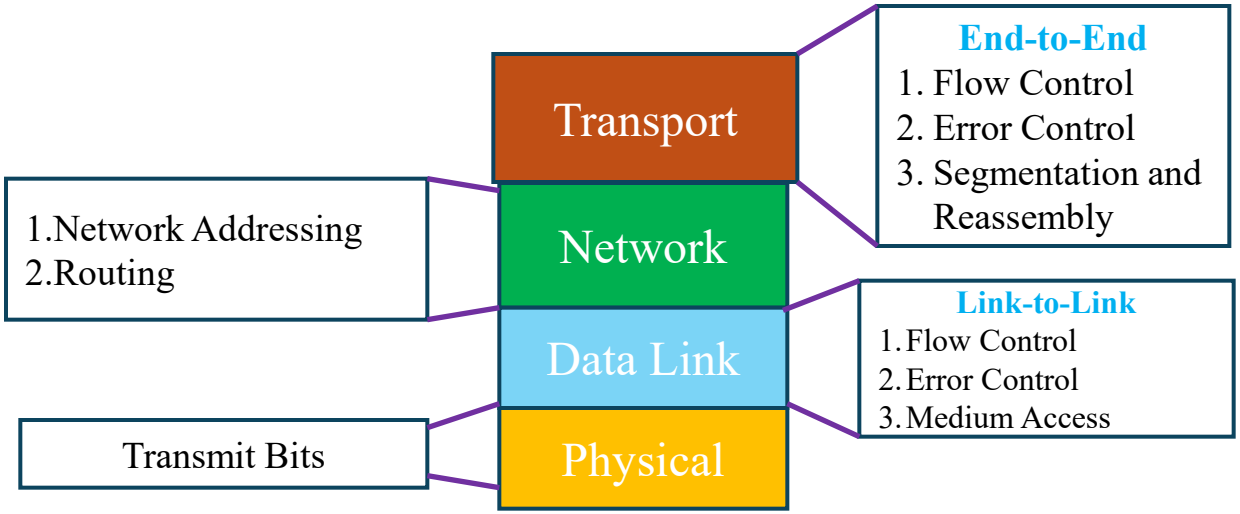
- Design issues,
- Routing algorithms: shortest path routing, Flooding, Hierarchical routing, Broadcast, Multicast, distance vector routing,
- Congestion Control Algorithms, Quality of Service,
- Internetworking, The Network layer in the internet.

## Reference:


1. Computer Networks, 5th Edition by Andrew Tanenbaum & David Wetherall, Copyright. Pearson Education 2011.
2. Jim Kurose, Keith Ross, Computer Networking: A Top-Down Approach, 8th Edition, Pearson Education.

# OSI Layers & Functionalities

Refresh



# **OSI L3 –** Network Layer

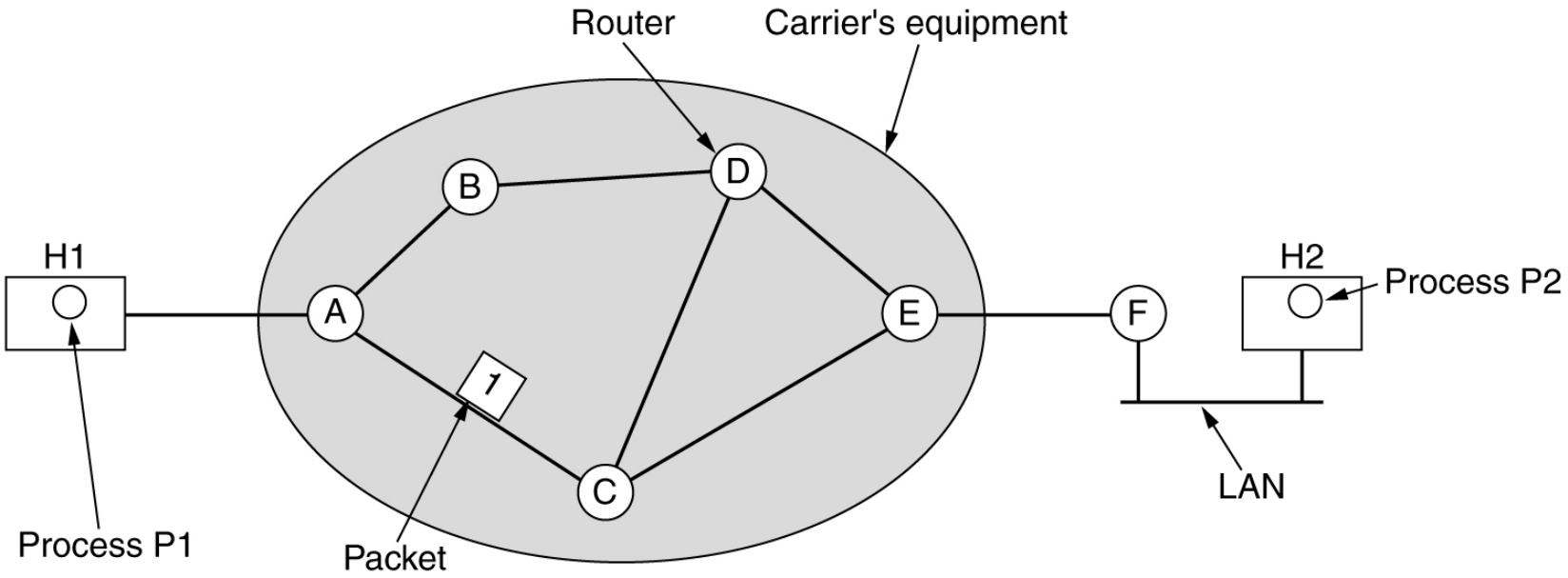
Refresh

- Responsible for addressing messages across networks
- Translates network logical addresses into physical machine addresses
- Breaks large segments into smaller packets
- Determines routing across networks
- Supports communications between logically separate networks
    - Routers and gateways (devices that pass data from one network to another) operate in the Network layer

# 1. Network Layer Design Issues

1. Store-and-Forward Packet Switching
2. Services Provided to the Transport Layer
    - Connectionless Service
    - Connection-Oriented Service

# Store-and-Forward Packet Switching



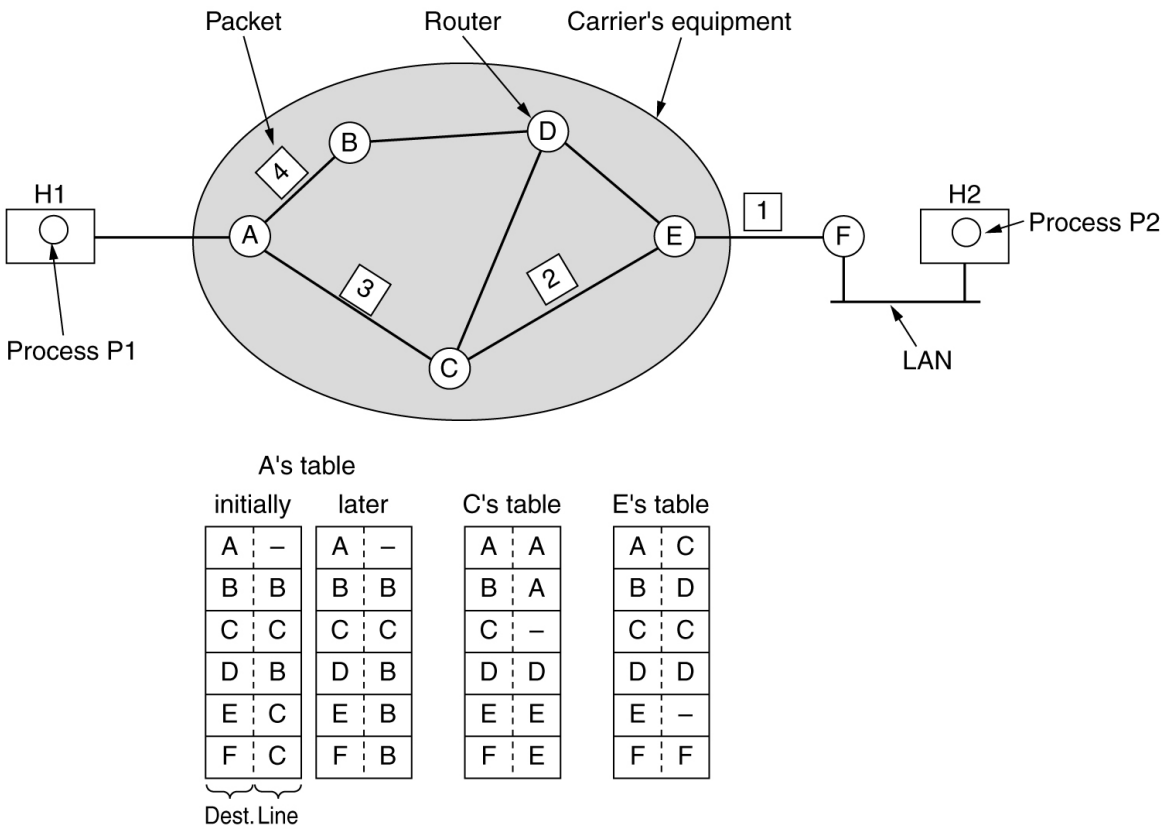

The environment of the network layer protocols.

# Services Provided to Transport Layer

The services need to be carefully designed with the following goals in mind:

1. The services should be independent of the router technology.
2. The transport layer should be shielded from the number, type, and topology of the routers present.
3. The network addresses made available to the transport layer should use a uniform numbering plan, even across LANs and WANs.

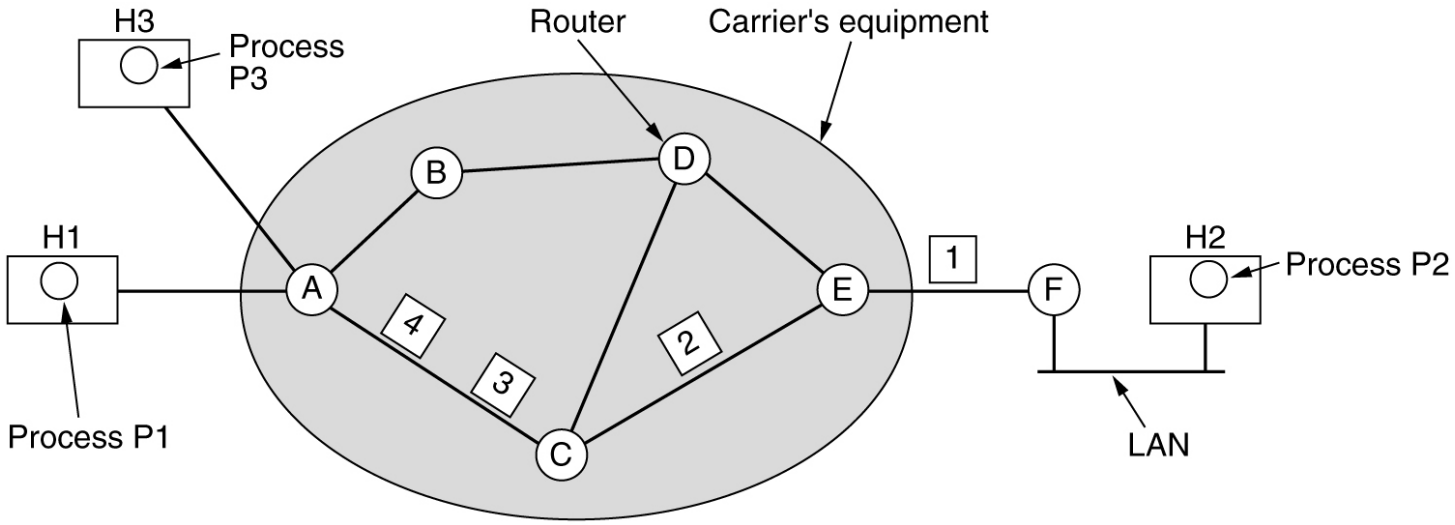
# Connectionless Service



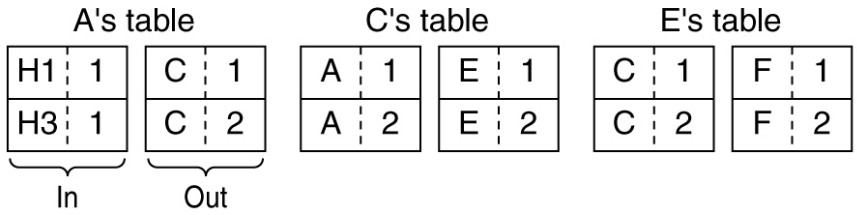

| A's table initially | | A's table later | | C's table | | E's table | |
|---|---|---|---|---|---|---|---|
| A | – | A | – | A | A | A | C |
| B | B | B | B | B | A | B | D |
| C | C | C | C | C | – | C | C |
| D | B | D | B | D | D | D | D |
| E | C | E | B | E | E | E | – |
| F | C | F | B | F | E | F | F |

Routing within a diagram subnet.

# Connection-Oriented Service




Label Switching



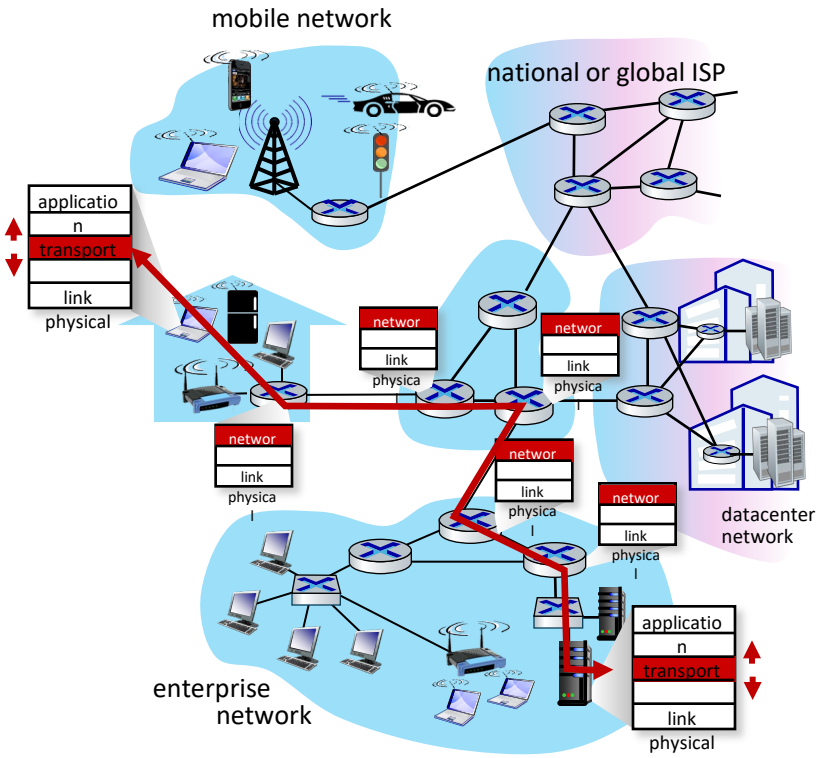

Routing within a virtual-circuit subnet.

# Connectionless VS. Connection-Oriented

| Issue | Datagram subnet | Virtual-circuit subnet |
|---|---|---|
| Circuit setup | Not needed | Required |
| Addressing | Each packet contains the full source and destination address | Each packet contains a short VC number |
| State information | Routers do not hold state information about connections | Each VC requires router table space per connection |
| Routing | Each packet is routed independently | Route chosen when VC is set up; all packets follow it |
| Effect of router failures | None, except for packets lost during the crash | All VCs that passed through the failed router are terminated |
| Quality of service | Difficult | Easy if enough resources can be allocated in advance for each VC |
| Congestion control | Difficult | Easy if enough resources can be allocated in advance for each VC |

# Network-layer services and protocols

- transport segment from sending to receiving host
  - sender: encapsulates segments into datagrams, passes to link layer
  - receiver: delivers segments to transport layer protocol
- network layer protocols in *every Internet device*: hosts, routers
- routers:
  - examines header fields in all IP datagrams passing through it
  - moves datagrams from input ports to output ports to transfer datagrams along end-end path



# ROUTING ALGORITHMS

1. Non-adaptive
   - do not base their routing decisions on any measurements or estimates of the current topology and traffic.
     - Static routing-the choice of the route to use to get from *I* to *J* (for all *I* and *J*) is computed in advance, offline, and downloaded to the routers when the network is booted.
2. Adaptive algorithms-
   - change their routing decisions to reflect changes in the topology, and sometimes changes in the traffic as well. **dynamic routing** algorithms differ in where they get their information (e.g.,locally, from adjacent routers, or from all routers), when they change the routes(e.g., when the topology changes, or every $\Delta T$ seconds as the load changes), and what metric is used for optimization (e.g., distance, number of hops, or estimatedtransit time).

Certain properties are desirable in a routing algorithm:

- Correctness,
- Simplicity,
- Robustness,
- Stability,
- Fairness, and
- Efficiency.

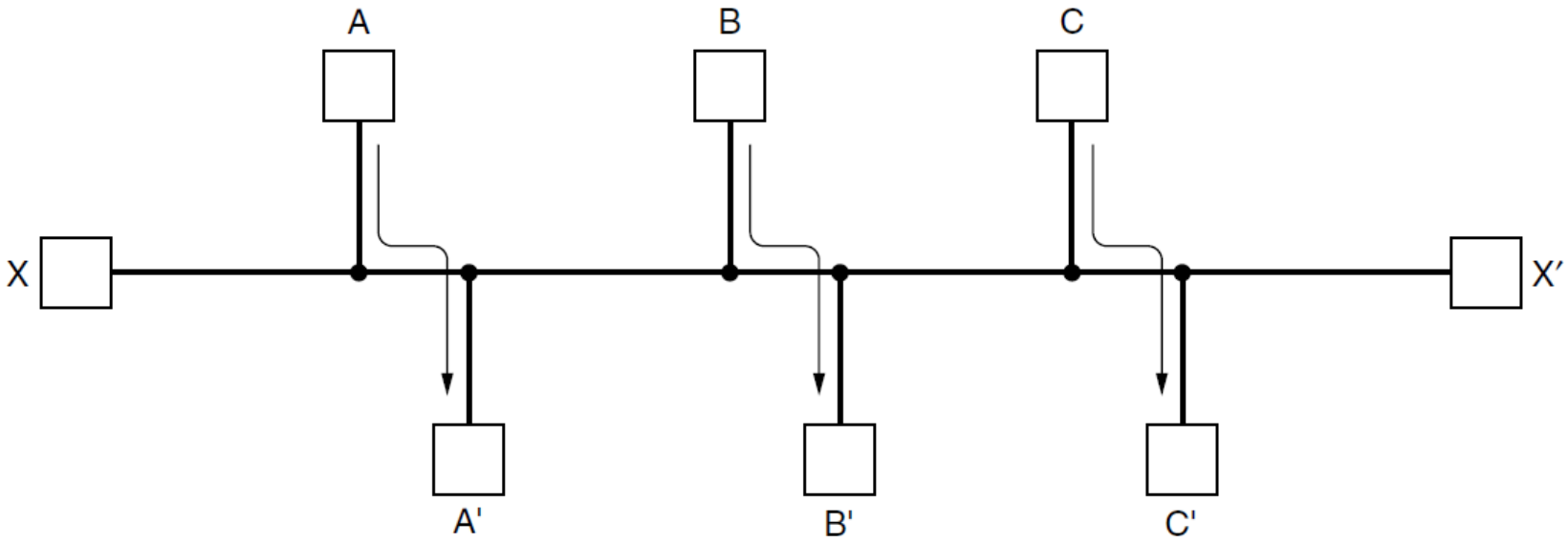



**Figure 5-5.** Network with a conflict between fairness and efficiency.

# The Optimality Principle

- general statement about optimal routes without regard to network topology or traffic.
- Bellman,1957
- It states that if router *J* is on the optimal path from router *I* to router *K*, then the optimal path from *J* to *K* also falls along the same route.

# The Optimality Principle

As a direct consequence of the optimality principle, we can see that the **set of optimal routes from all sources to a given destination form a tree rooted at the destination.** Such a tree is called a **sink tree** and is illustrated in Fig. 5-6(b), where the distance metric is the number of hops. The goal of all routing algorithms is to discover and use the sink trees for all routers.

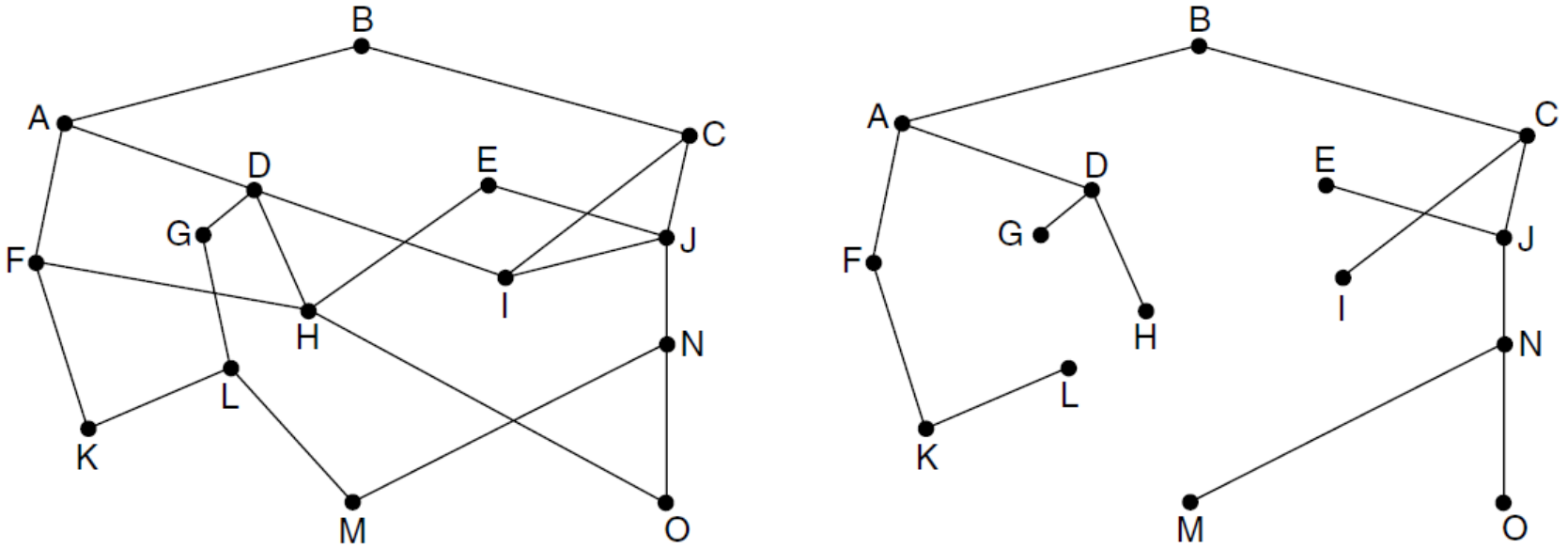
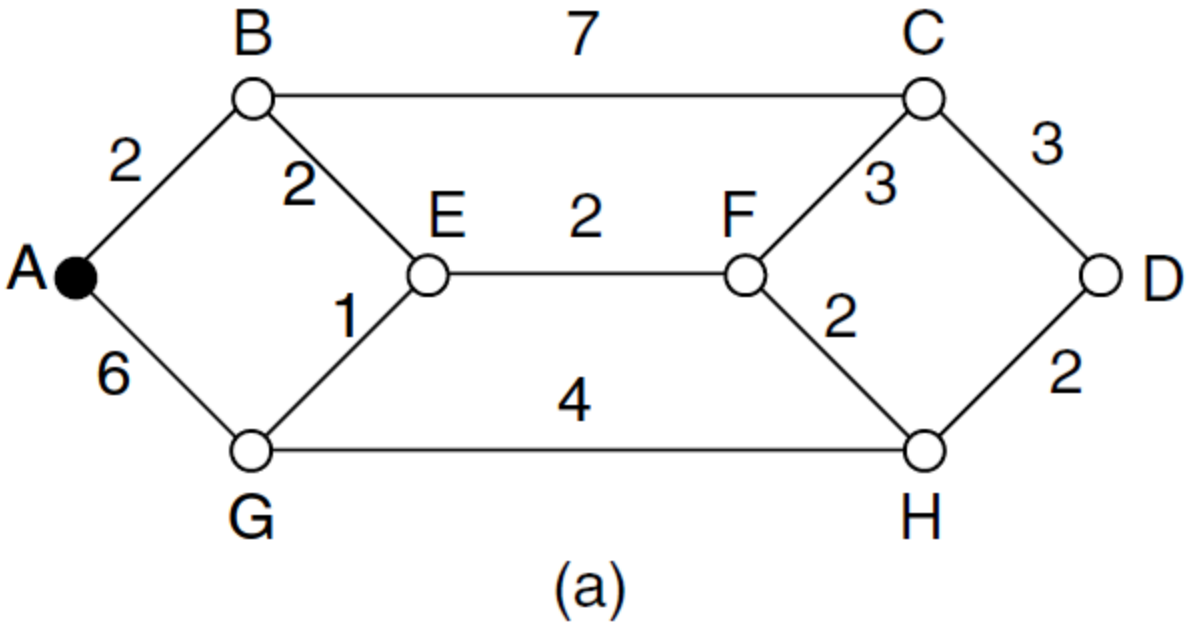



**Figure 5-6.** (a) A network. (b) A sink tree for router *B*.

# Shortest Path Algorithm



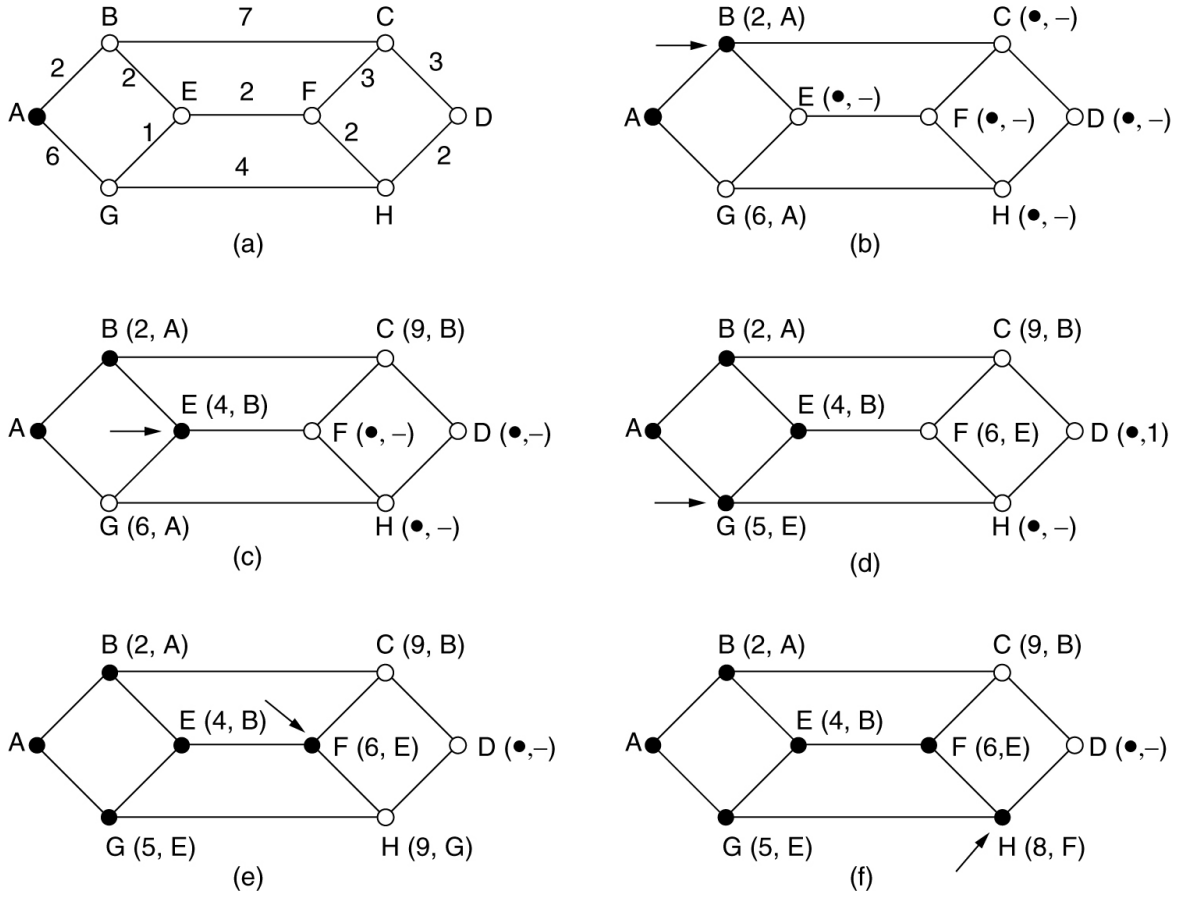

(a)

# Shortest Path Algorithm

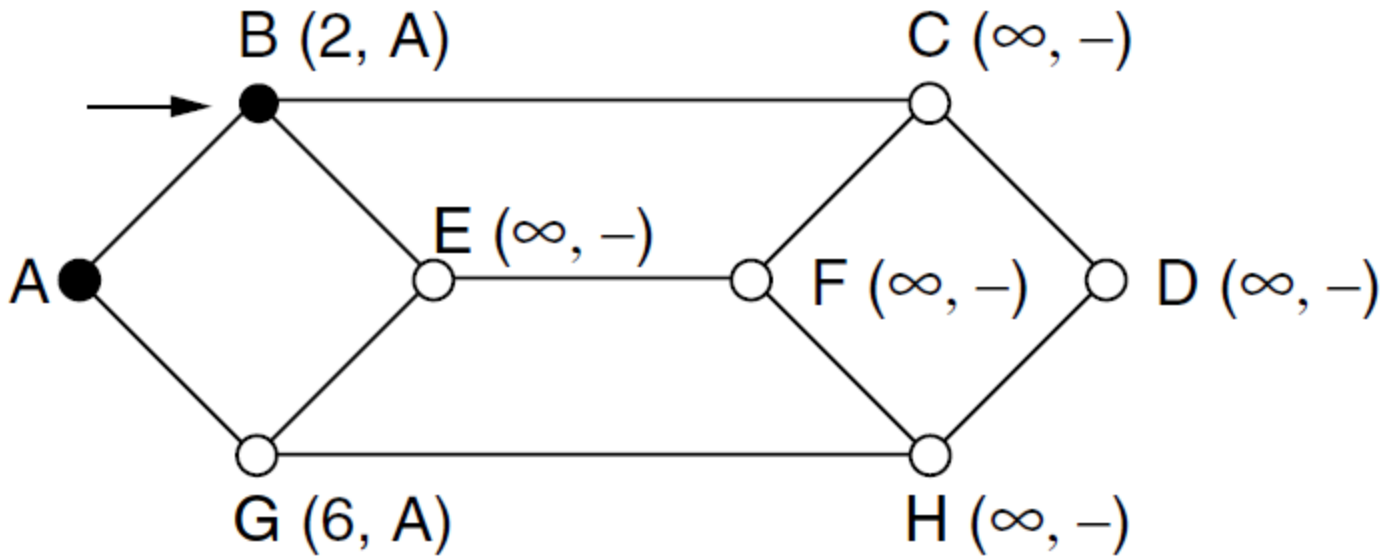
In the general case, the labels on the edges could be computed as a function of the

- distance,
- bandwidth,
- average traffic,
- communication cost,
- measured delay,
- and other factors.
- By changing the weighting function, the algorithm would then compute the “shortest’’ path measured according to any one of a number of criteria or to a combination of criteria.

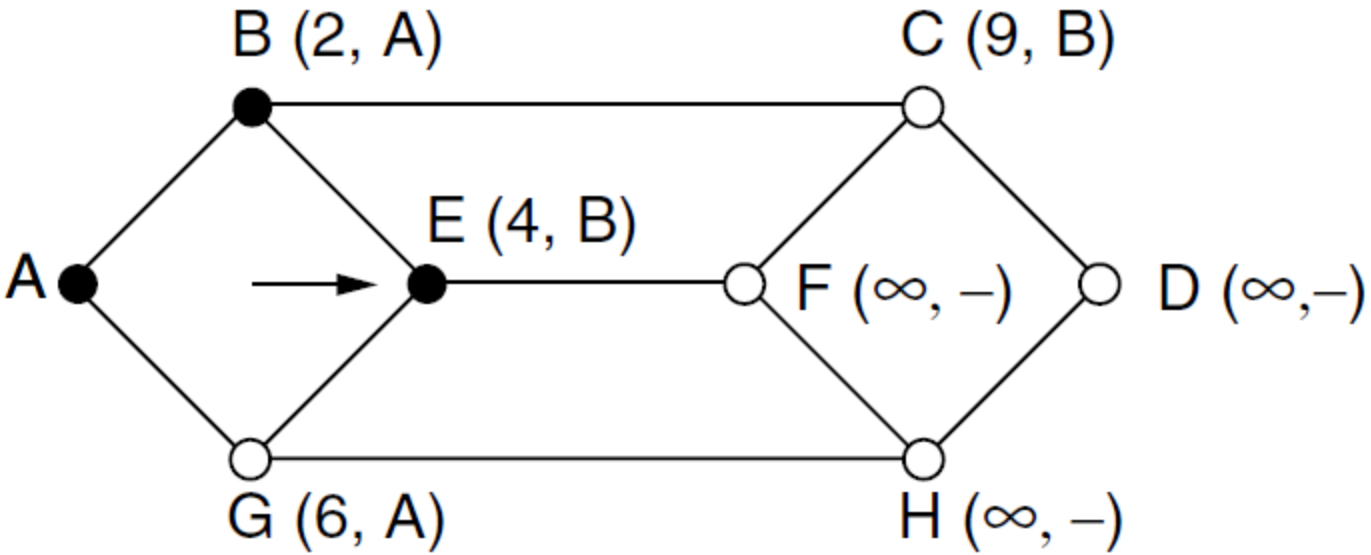
# Shortest Path Algorithm



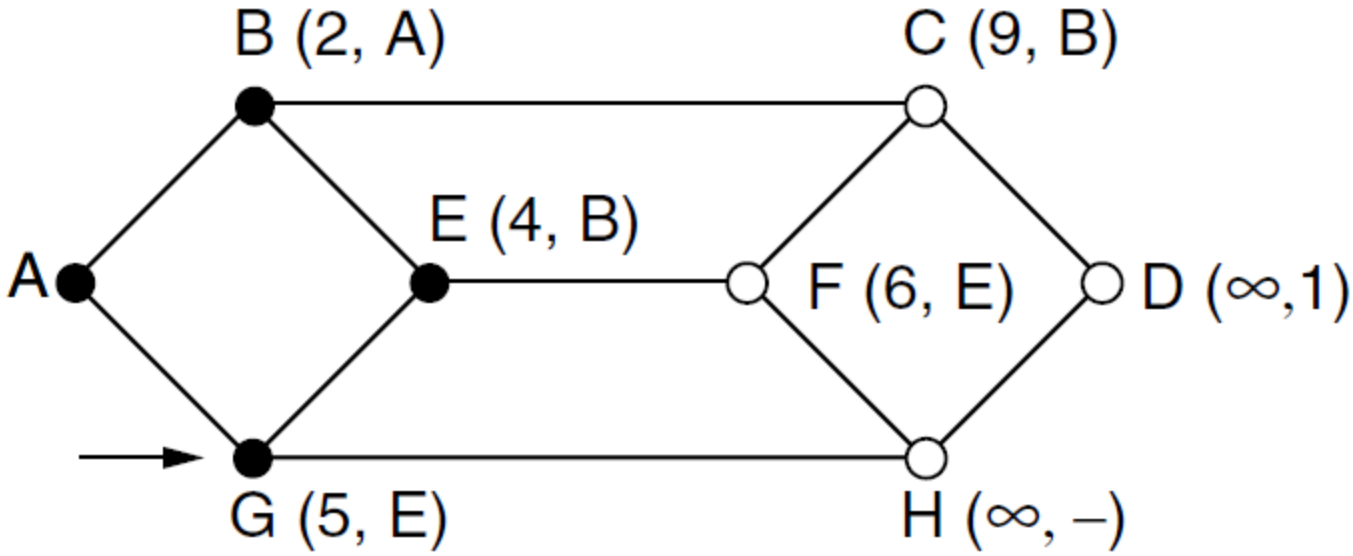

- The first 5 steps used in computing the shortest path from A to D. The arrows indicate the working node.



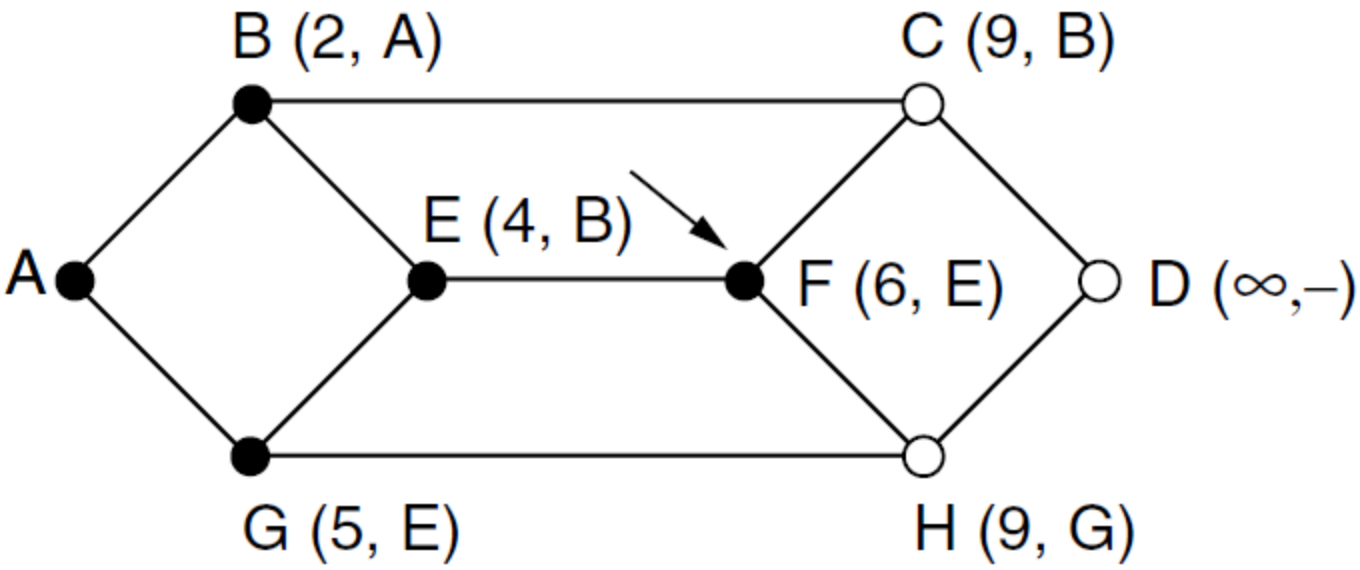

(b)



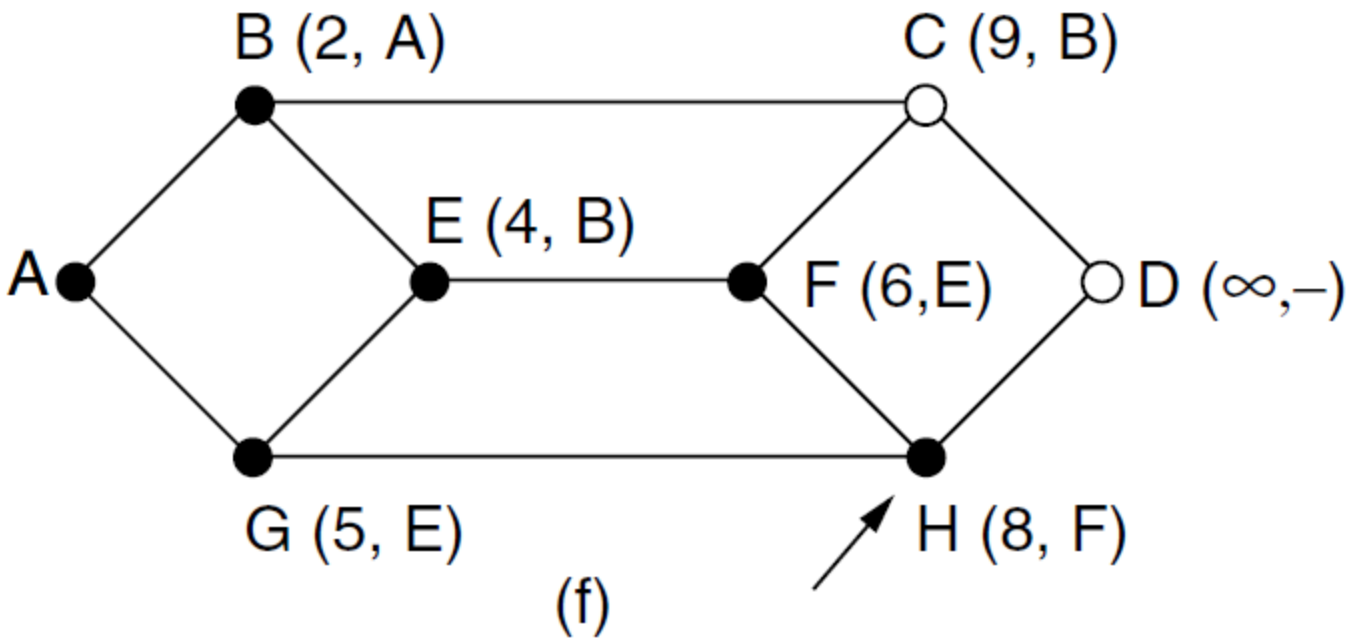

(c)




(d)




(e)

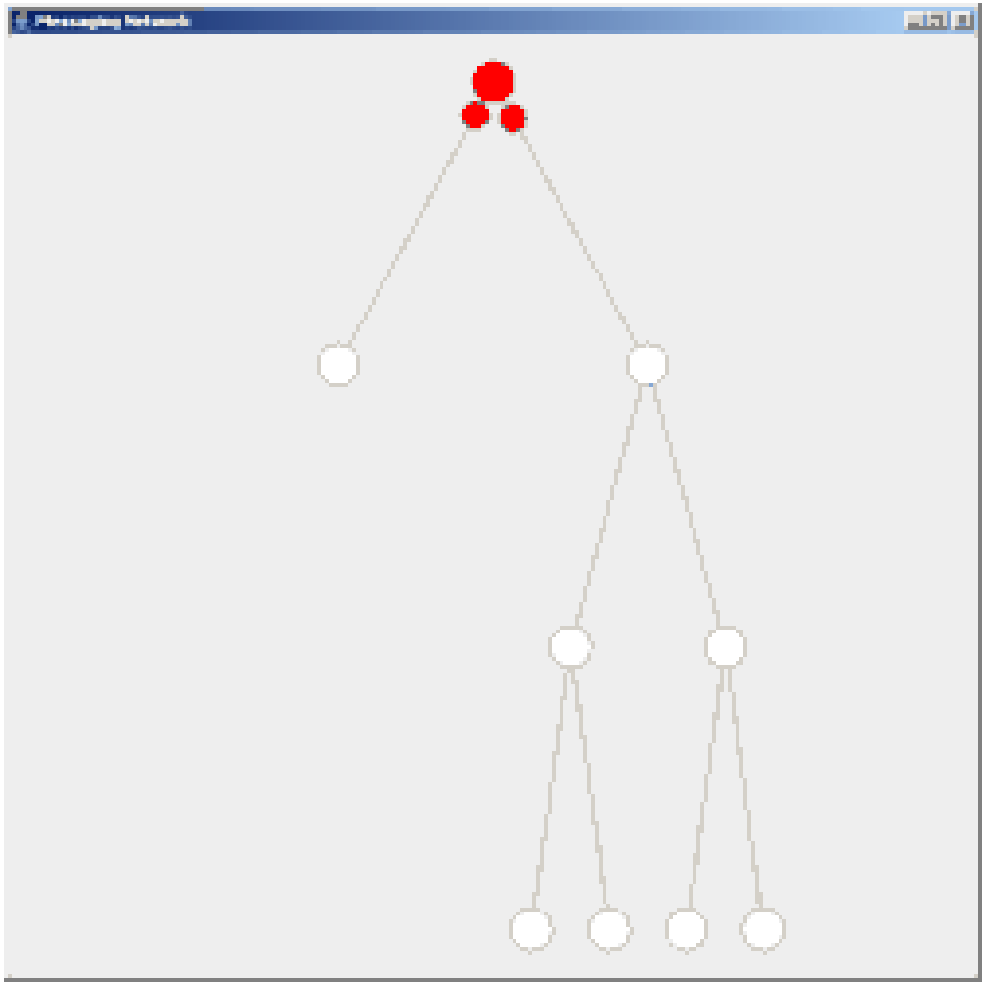



(f)

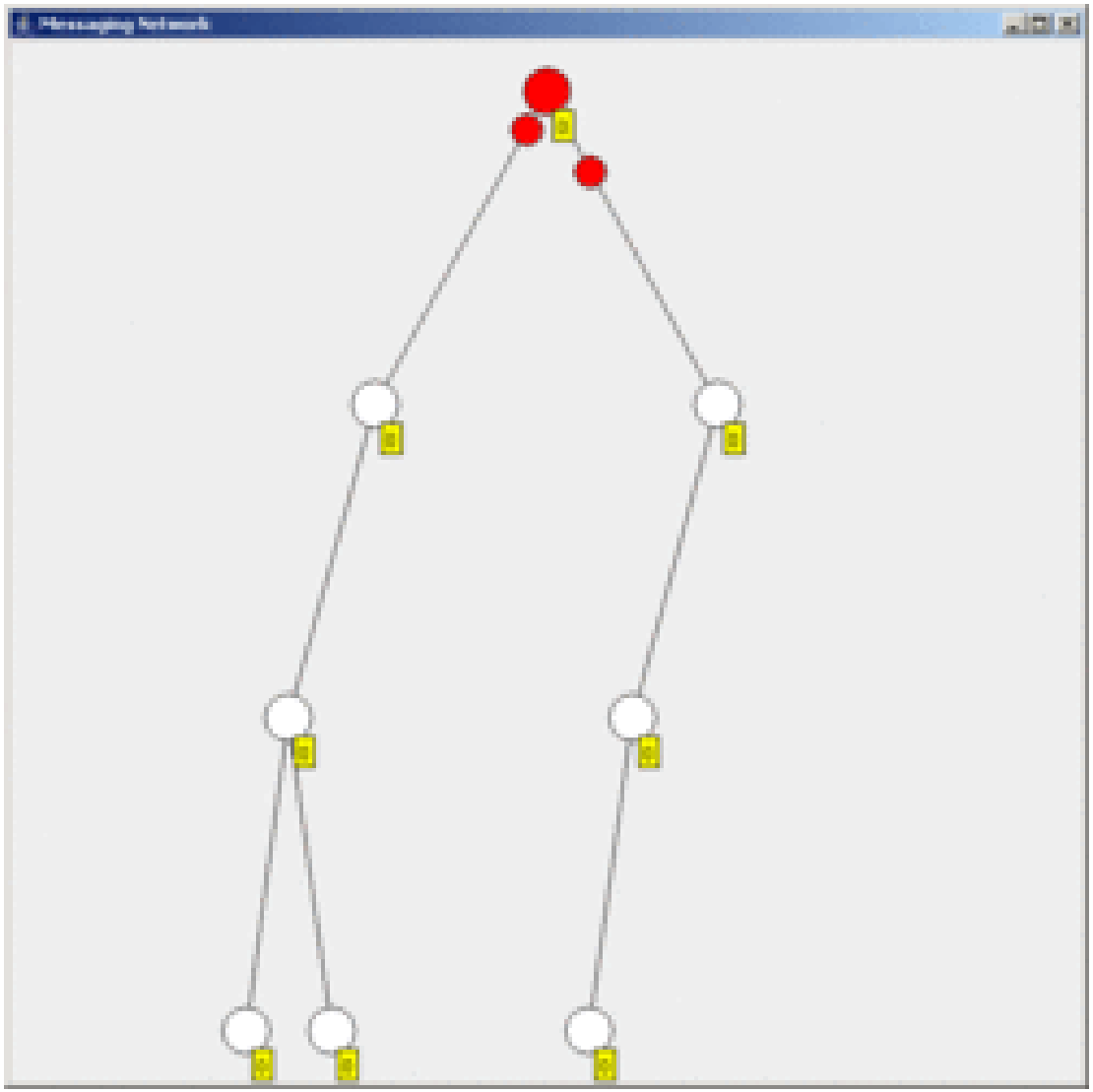
# Flooding

•**Definition**: Forward incoming packets out all other links—no routing logic applied.

- is a routing technique where a packet is sent out on every outgoing link of a router, except the one it arrived on.
- This ensures the packet reaches all nodes in the network, making it useful for finding paths and distributing information.
- However, it can also lead to network congestion due to duplicate packets



# Flooding



# Why Use Flooding?

- **Robustness**: Delivers to all reachable nodes even under failures.
- **Simplicity**: No pre-established routes needed.
- **Shortest Path**: Utilizes all possible routes, effectively the fastest path.

# Flooding Caveats

- Generates numerous **duplicate packets**
- Can flood the network excessively without safeguards
- **Not efficient** for typical datagram delivery

# Flooding

- **Network Discovery:** Flooding is used to discover all the nodes in a network when the network's topology is unknown or dynamic.
- **Routing Protocol Initialization:** Flooding helps initialize routing tables when network nodes are first brought online or when network topology changes.
- **Broadcasting Messages:** Flooding ensures that messages reach every part of a network, particularly in situations such as system updates, network announcements, or alerts.
- **Path Discovery in Routing Protocols:** In protocols such as Distance-Vector Routing and Link-State Routing, flooding enables nodes to discover the best route by distributing information about the network topology.
- **Multicast and Anycast Communication:** Flooding ensures that multicast and anycast messages are delivered to relevant nodes efficiently, especially in large, distributed networks.

# Types of Flooding

- **Uncontrolled flooding** Here, each router unconditionally transmits the incoming data packets to all its neighbours.

- **Controlled flooding** They use some methods to control the transmission of packets to the neighbouring nodes. The two popular algorithms for controlled flooding are Sequence Number Controlled Flooding (SNCF) and Reverse Path Forwarding (RPF).

- **Selective flooding** Here, the routers don't transmit the incoming packets only along those paths which are heading towards approximately in the right direction, instead of every available paths.

## Damming

- Flooding with a hop count can produce an exponential number of duplicate packets as the hop count grows and routers duplicate packets they have seen before.

- A better technique for damming the flood is to have routers keep track of which packets have been flooded, to avoid sending them out a second time.

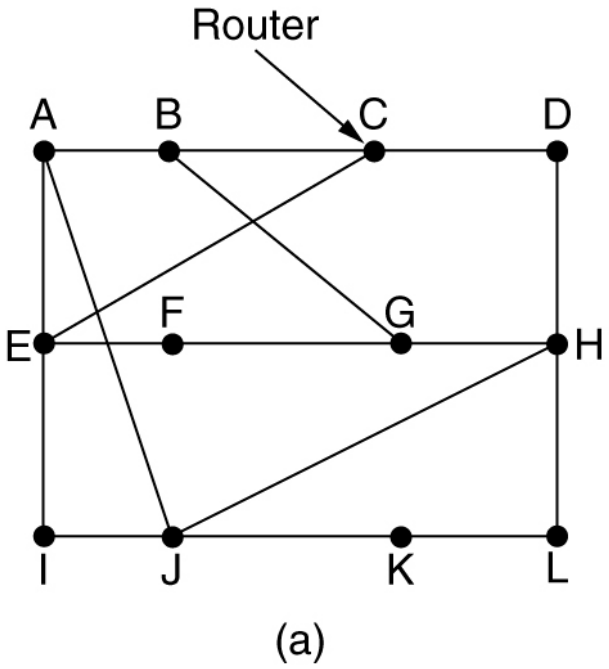
# Mitigation Techniques

- **Sequence Numbers / Memory**: Drop previously seen packets
- **Hop Count / TTL**: Restrict packet lifetime
- **Reliable Flooding**: Use age and acknowledgements to manage replication (used in link-state protocols)

# Distance Vector Routing

- Distributed Bellman–Ford-based routing algorithm
- Routers maintain and share distance vectors (best-known path + distance) with neighbors

- **Mechanism**
- Periodic advertisement of routing tables
- Distance metric: hops or other costs
- Updates on receiving neighbor vectors
- Convergence time ≈ network diameter




(a)

New estimated delay from J

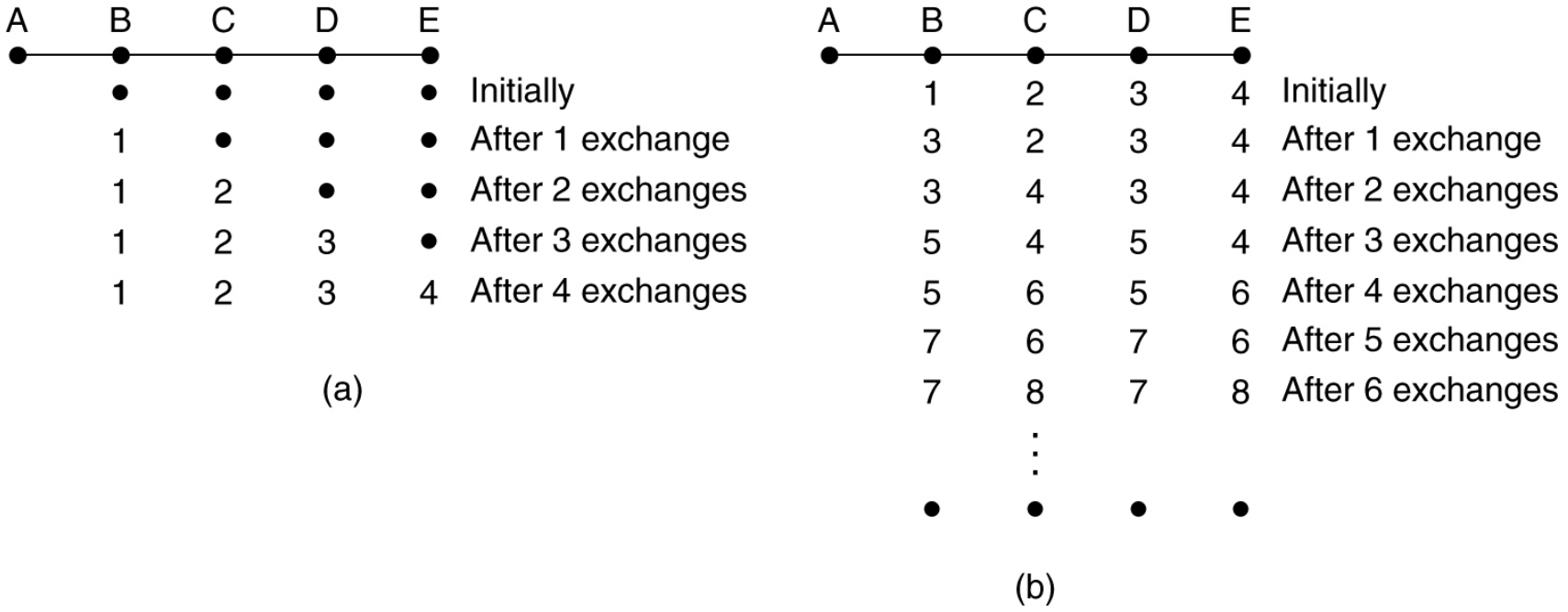
| To | A | I | H | K | Line |
|---|---|---|---|---|---|
| A | 0 | 24 | 20 | 21 | 8 A |
| B | 12 | 36 | 31 | 28 | 20 A |
| C | 25 | 18 | 19 | 36 | 28 I |
| D | 40 | 27 | 8 | 24 | 20 H |
| E | 14 | 7 | 30 | 22 | 17 I |
| F | 23 | 20 | 19 | 40 | 30 I |
| G | 18 | 31 | 6 | 31 | 18 H |
| H | 17 | 20 | 0 | 19 | 12 H |
| I | 21 | 0 | 14 | 22 | 10 I |
| J | 9 | 11 | 7 | 10 | 0 – |
| K | 24 | 22 | 22 | 0 | 6 K |
| L | 29 | 33 | 9 | 9 | 15 K |
| | JA delay is 8 | JI delay is 10 | JH delay is 12 | JK delay is 6 | New routing table for J |

Vectors received from J's four neighbors

(b)

(a) A subnet. (b) Input from A, I, H, K, and the new routing table for J.

# Distance Vector Routing

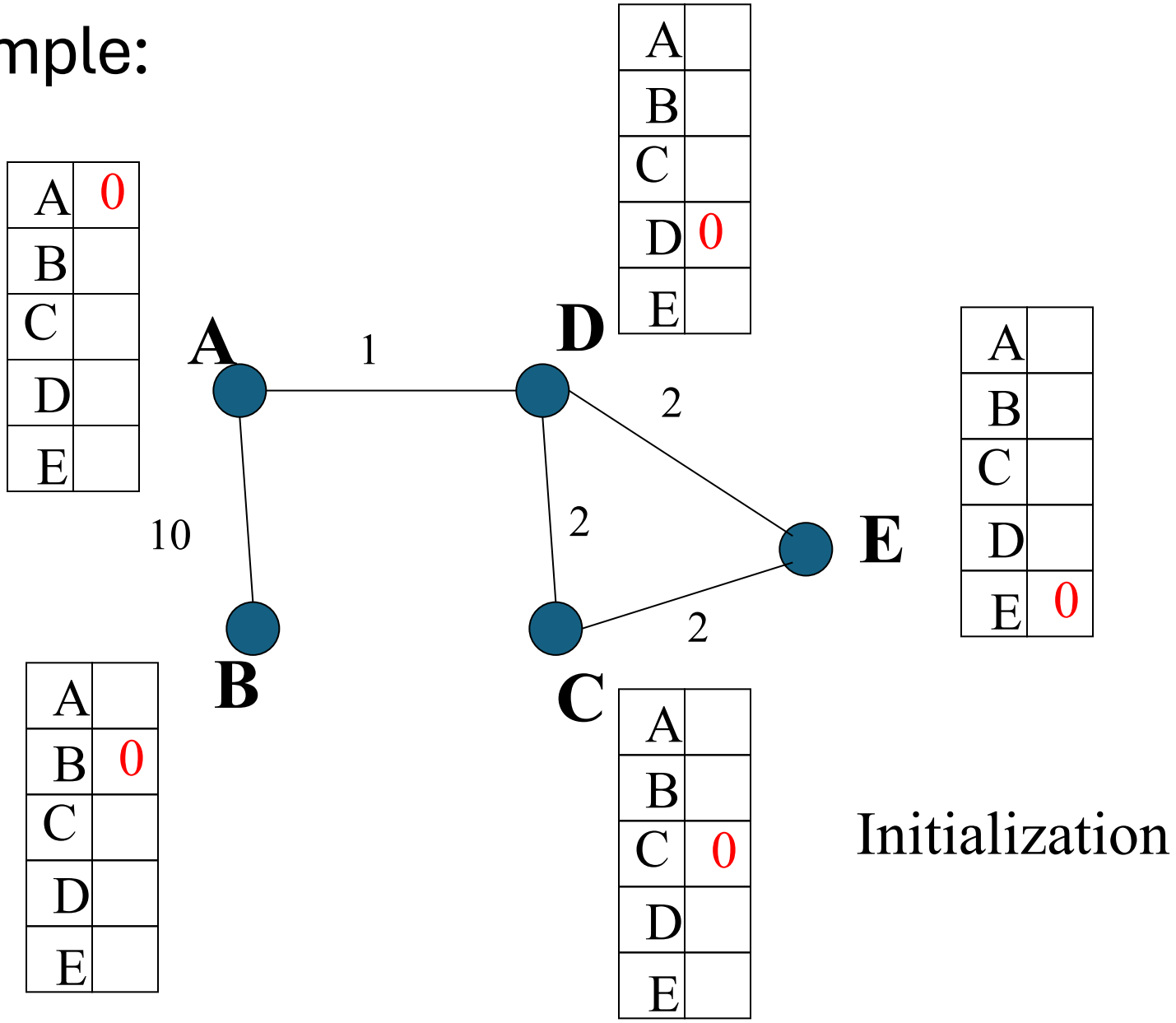


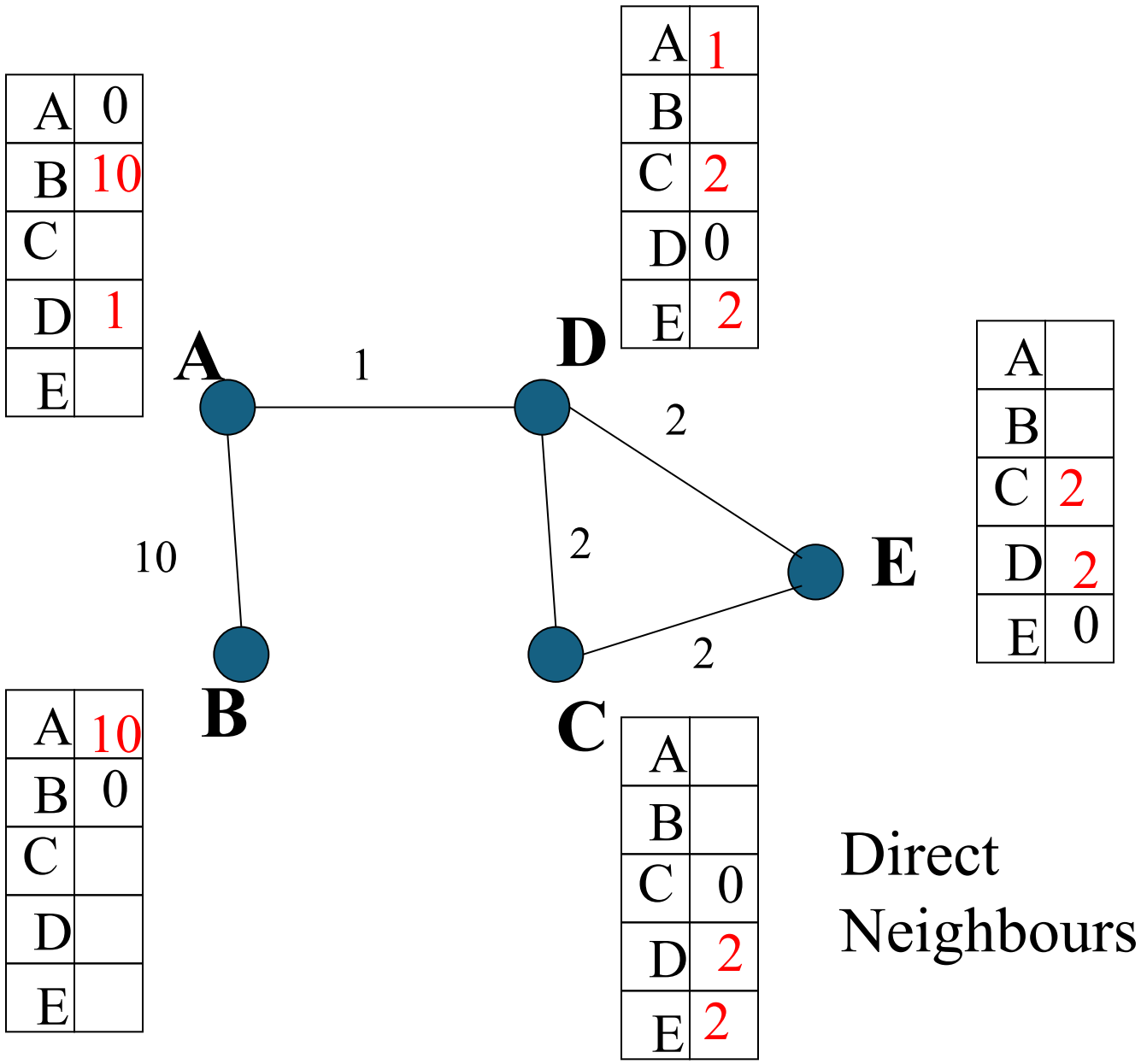

The count-to-infinity problem.

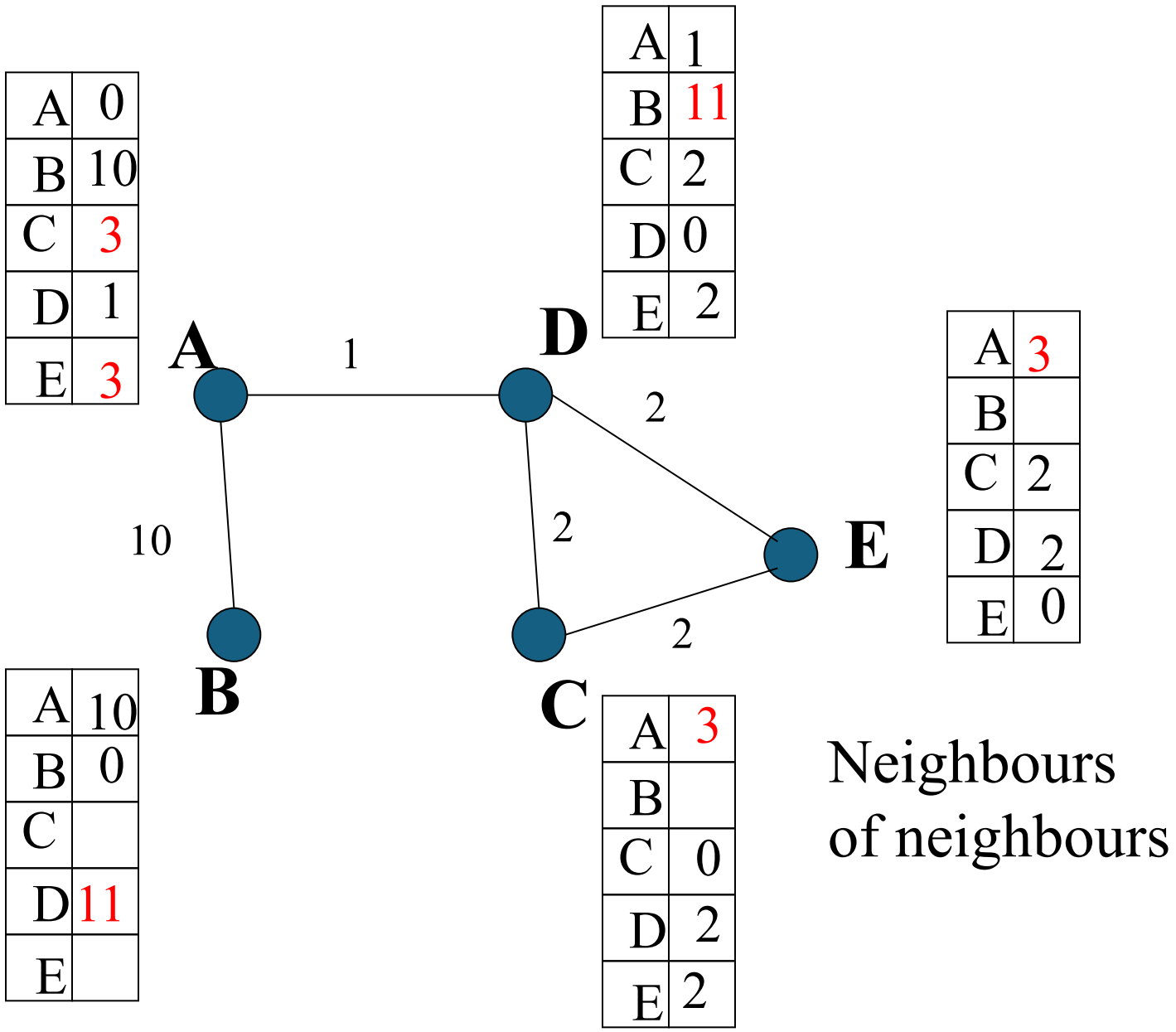
# Distance Vector Routing : Loop-Breaking Heuristics

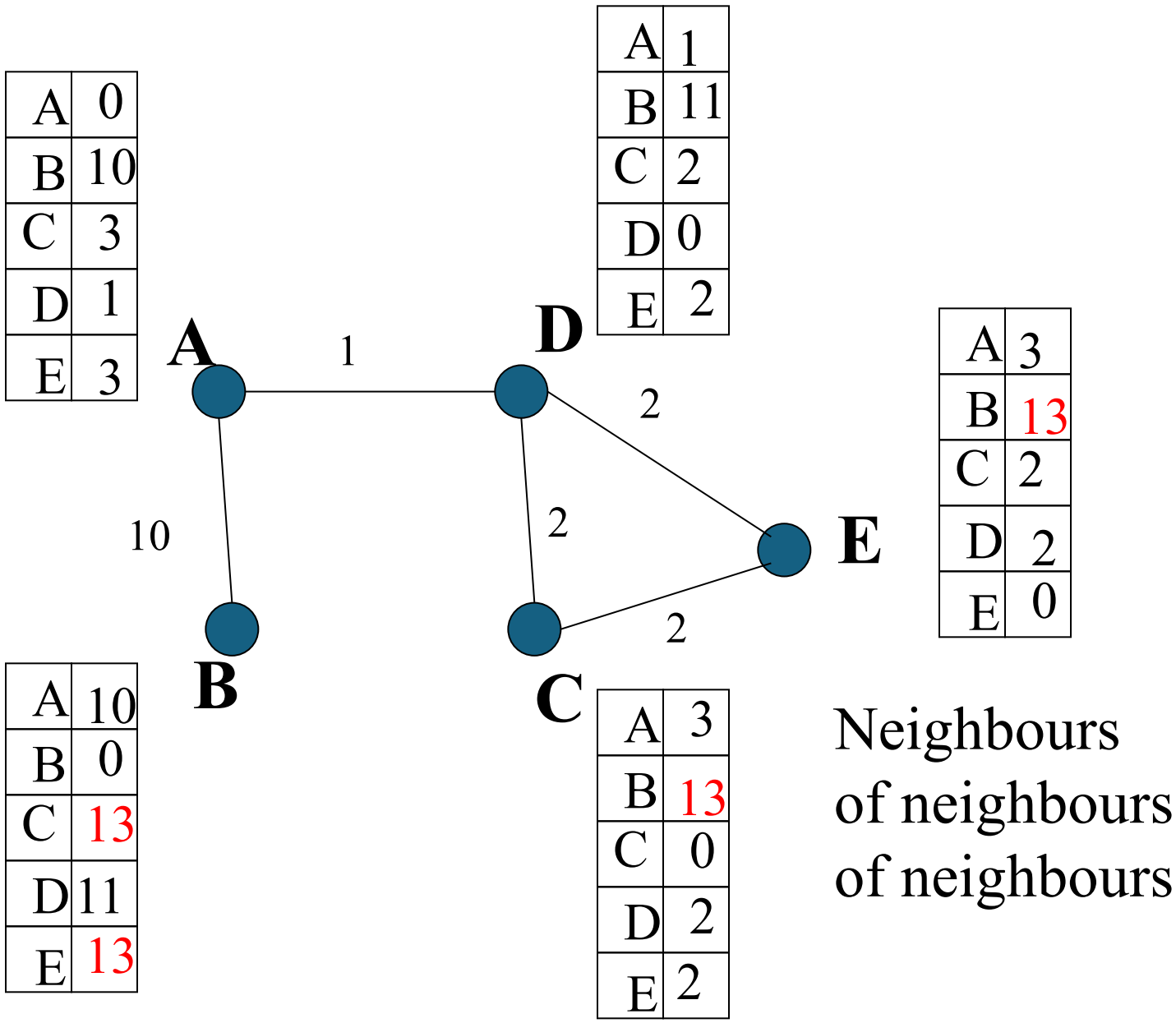
- Set infinity to a limited number, e.g. 16.
- Split horizon
- Split horizon with poison reverse

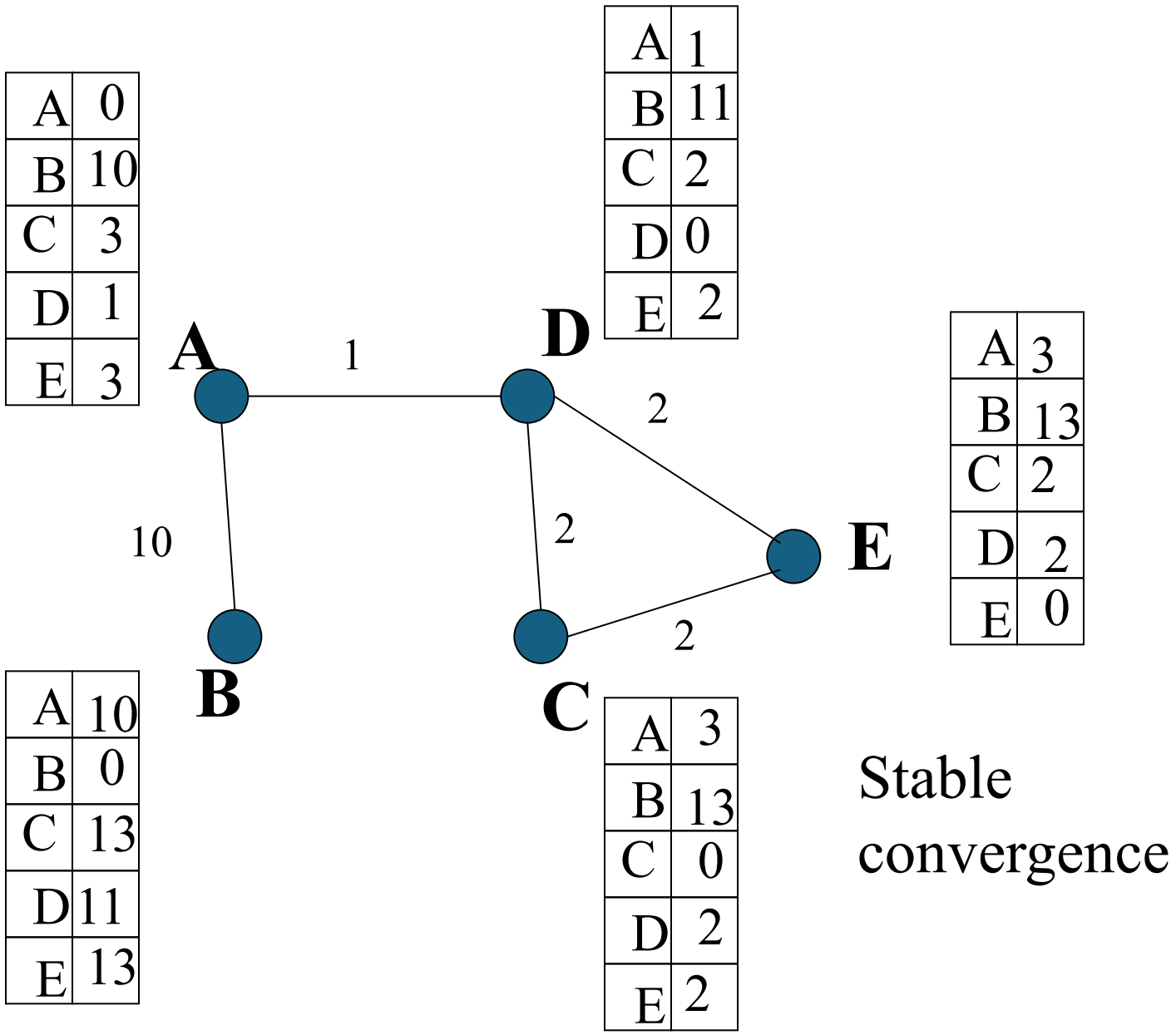
# Example:

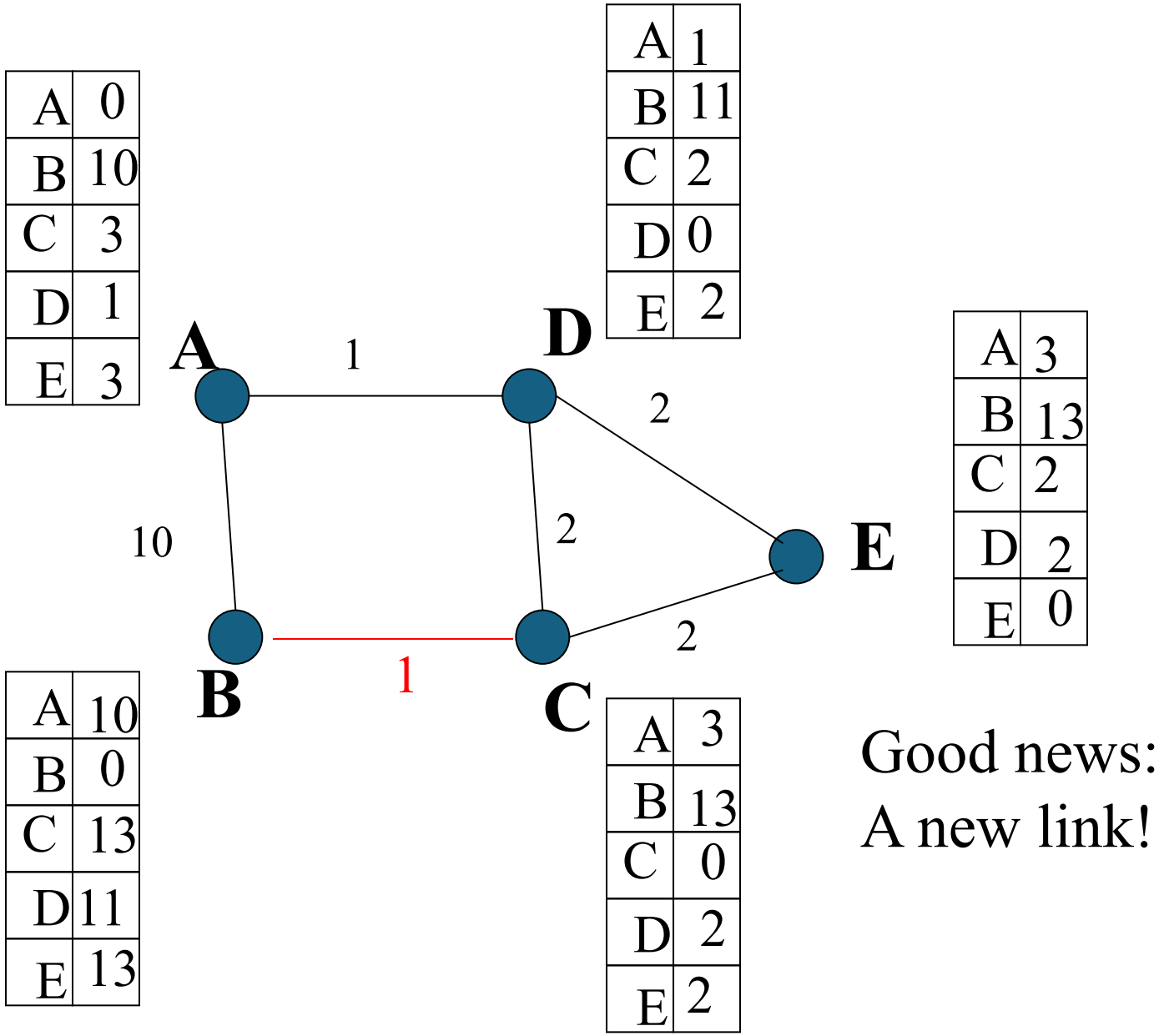

A
B
C
D
E
1
2
2
2
10
A 0
B
C
D
E
A
B
C
D 0
E
A
B
C
D
E 0
A
B 0
C
D
E
A
B
C 0
D
E
Initialization

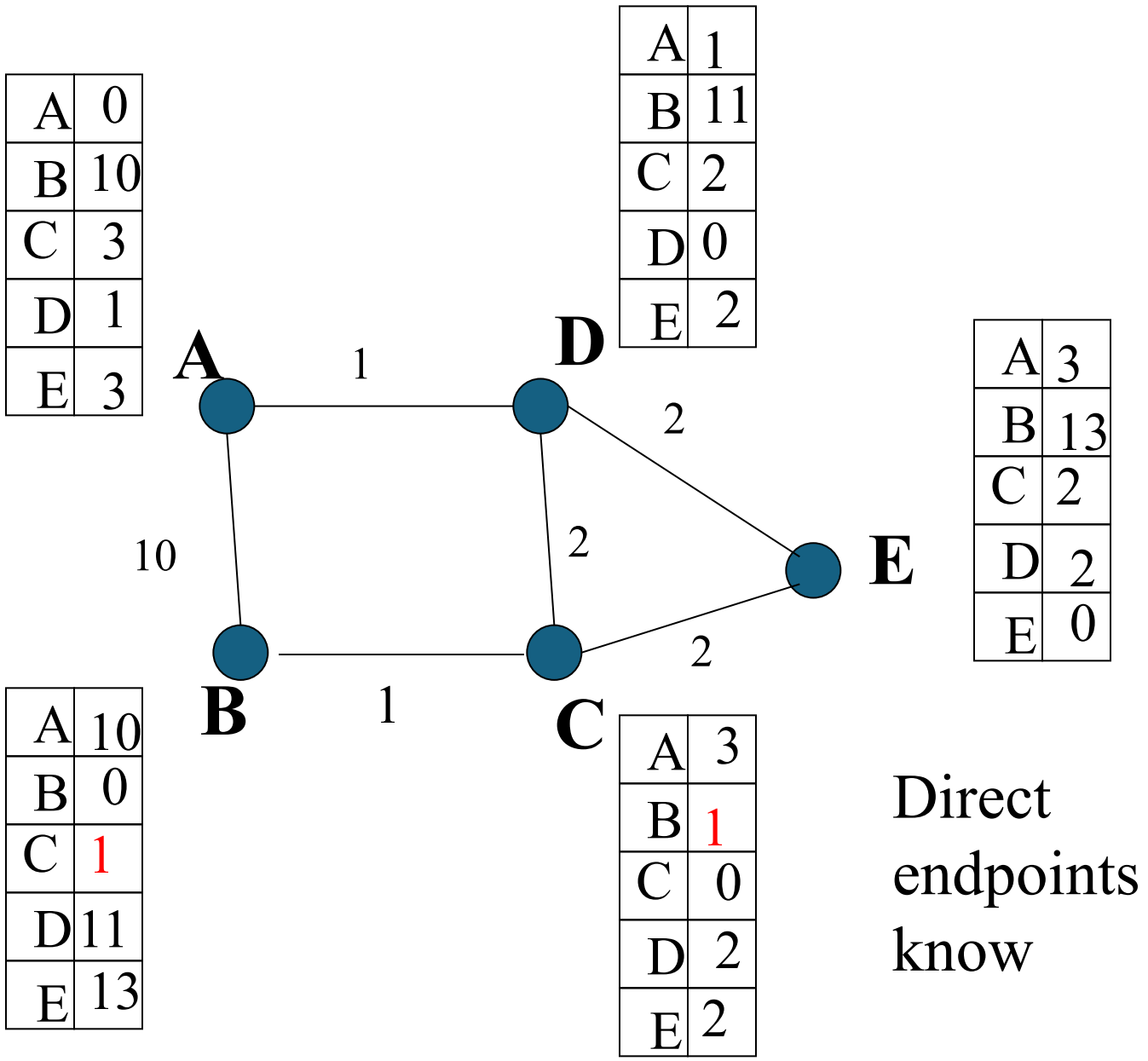

A
B
C
D
E
1
10
2
2
2
A 0
B 10
C
D 1
E
A 1
B
C 2
D 0
E 2
A
B
C 2
D 2
E 0
A 10
B 0
C
D
E
A
B
C 0
D 2
E 2
Direct
Neighbours

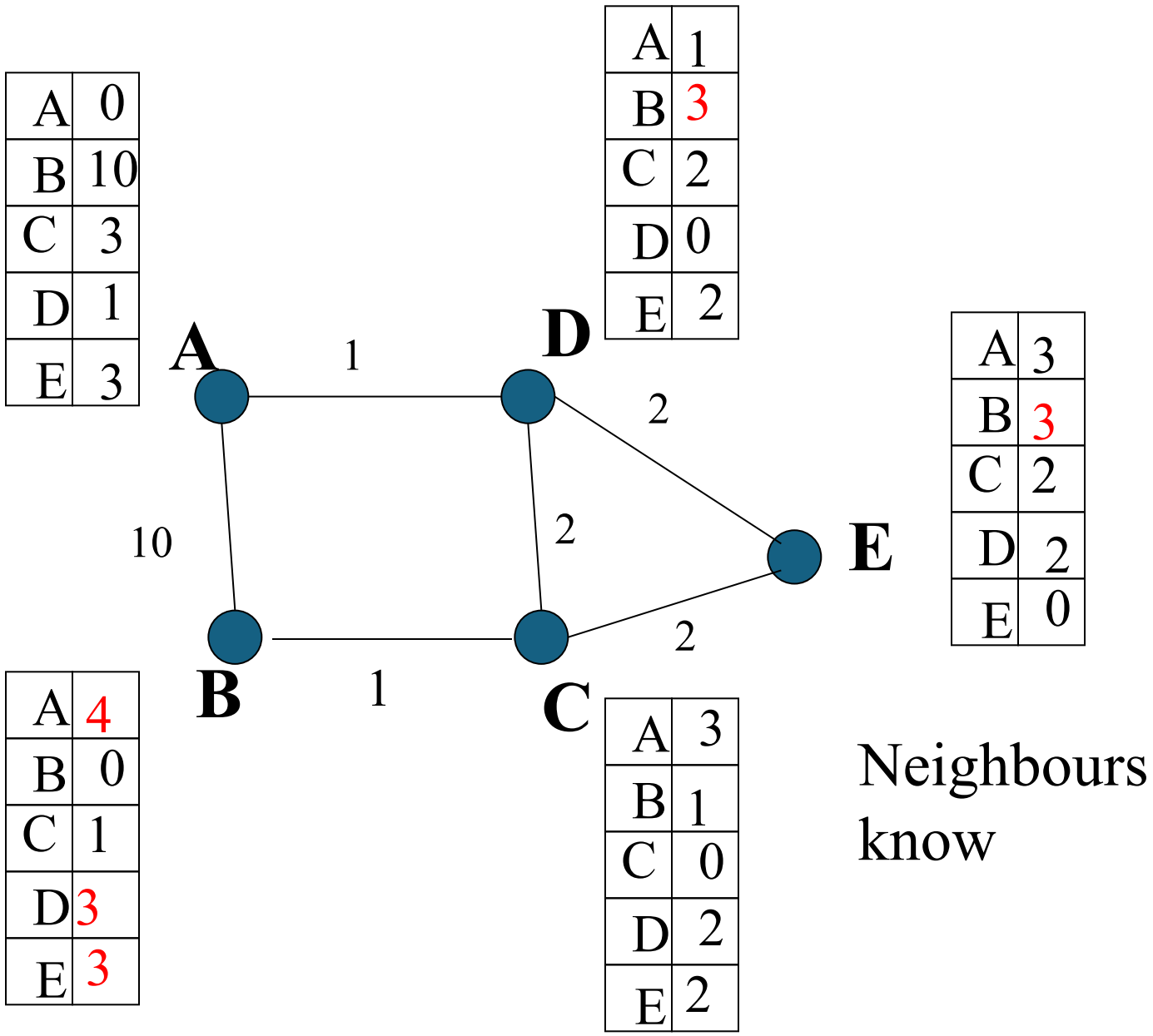

A
B
C
D
E
1
10
2
2
2
A 0
B 10
C 3
D 1
E 3
A 1
B 11
C 2
D 0
E 2
A 3
B
C 2
D 2
E 0
A 10
B 0
C
D 11
E
A 3
B
C 0
D 2
E 2
Neighbours
of neighbours

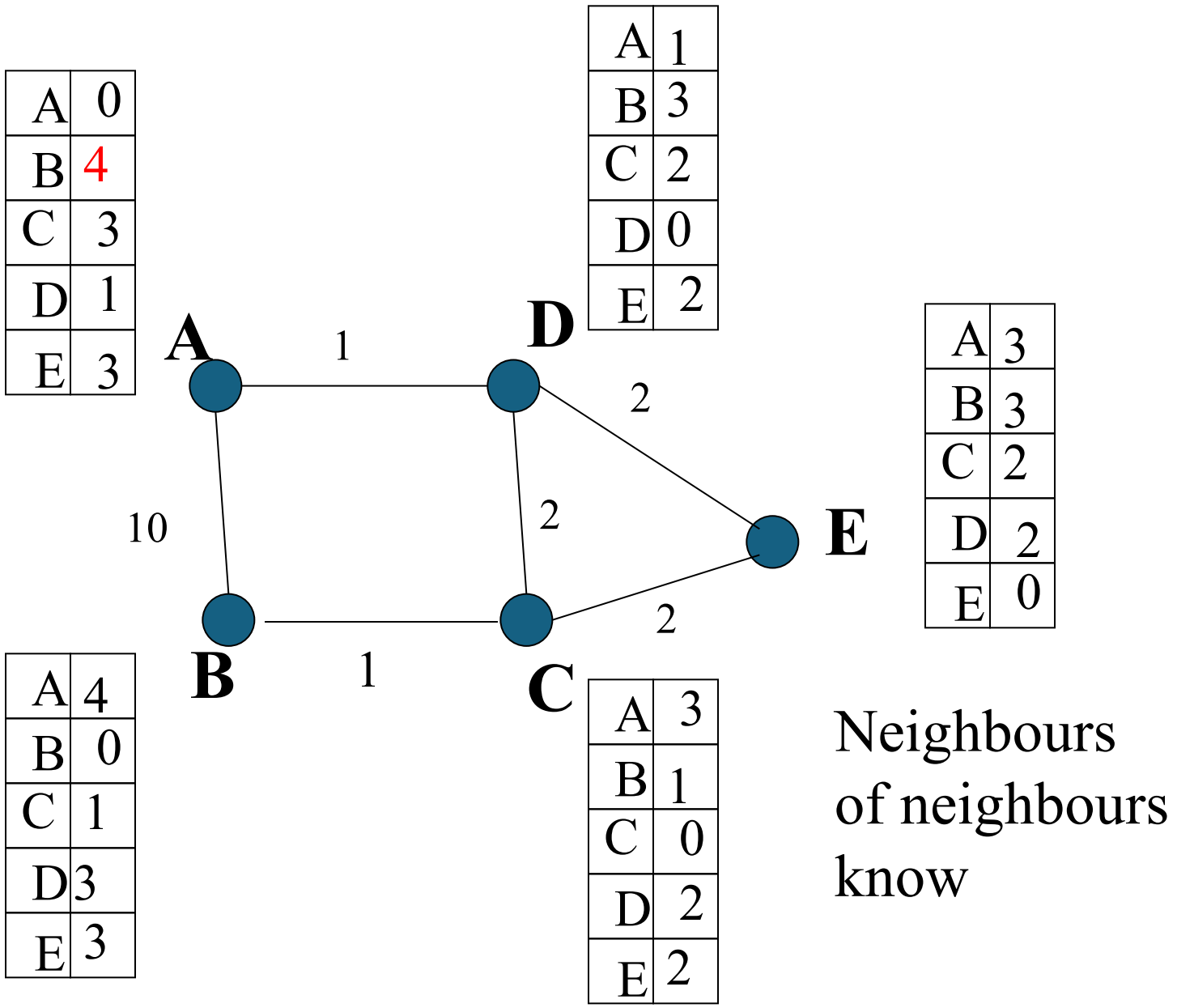

A
B
C
D
E
1
10
2
2
2
A 0
B 10
C 3
D 1
E 3
A 1
B 11
C 2
D 0
E 2
A 3
B 13
C 2
D 2
E 0
A 10
B 0
C 13
D 11
E 13
A 3
B 13
C 0
D 2
E 2
Neighbours
of neighbours
of neighbours

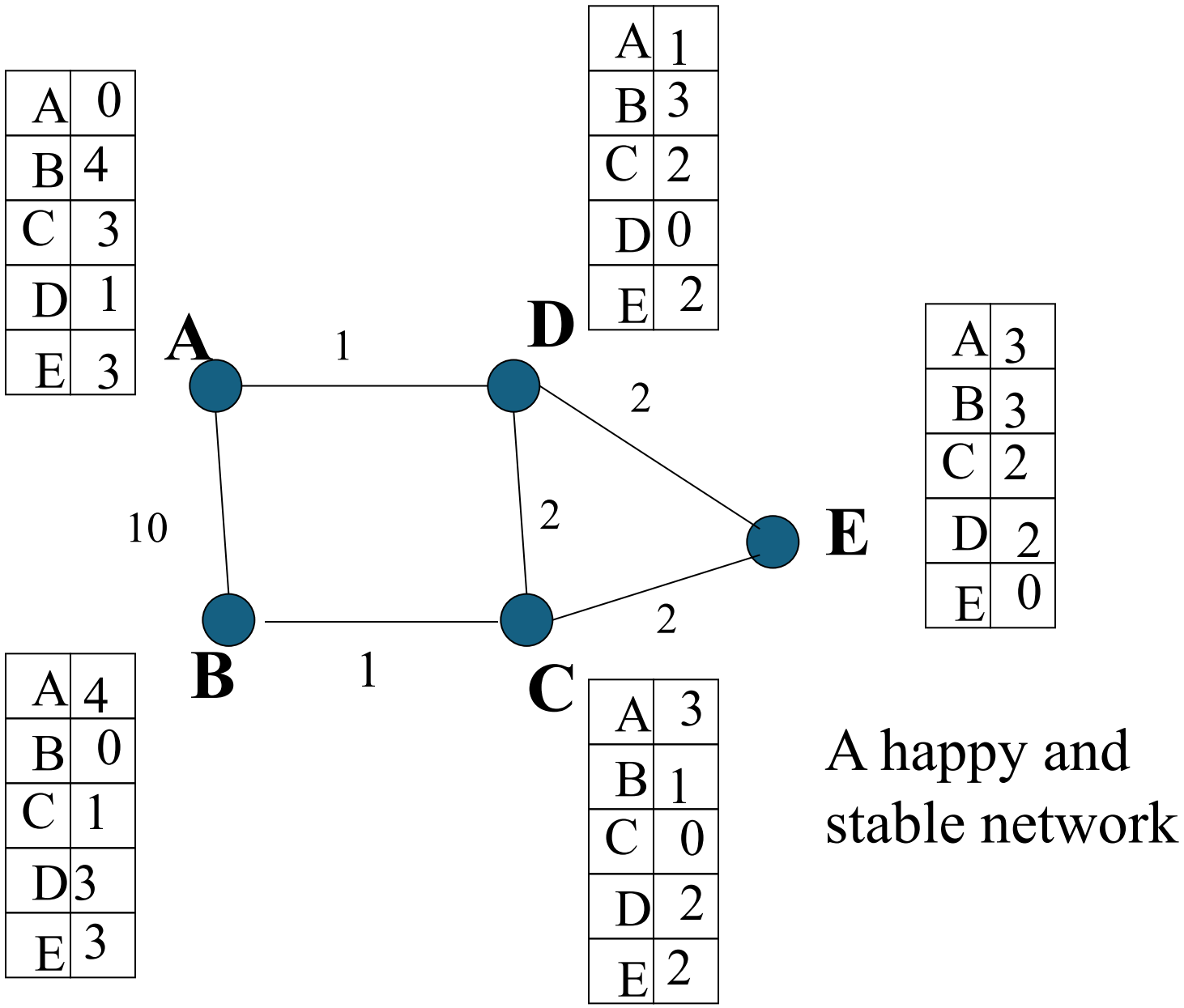

A
B
C
D
E
1
10
2
2
2
A 0
B 10
C 3
D 1
E 3
A 1
B 11
C 2
D 0
E 2
A 3
B 13
C 2
D 2
E 0
A 10
B 0
C 13
D 11
E 13
A 3
B 13
C 0
D 2
E 2
Stable convergence


A
B
C
D
E
1
10
2
2
2
1
A 0
B 10
C 3
D 1
E 3
A 1
B 11
C 2
D 0
E 2
A 3
B 13
C 2
D 2
E 0
A 10
B 0
C 13
D 11
E 13
A 3
B 13
C 0
D 2
E 2
Good news:
A new link!

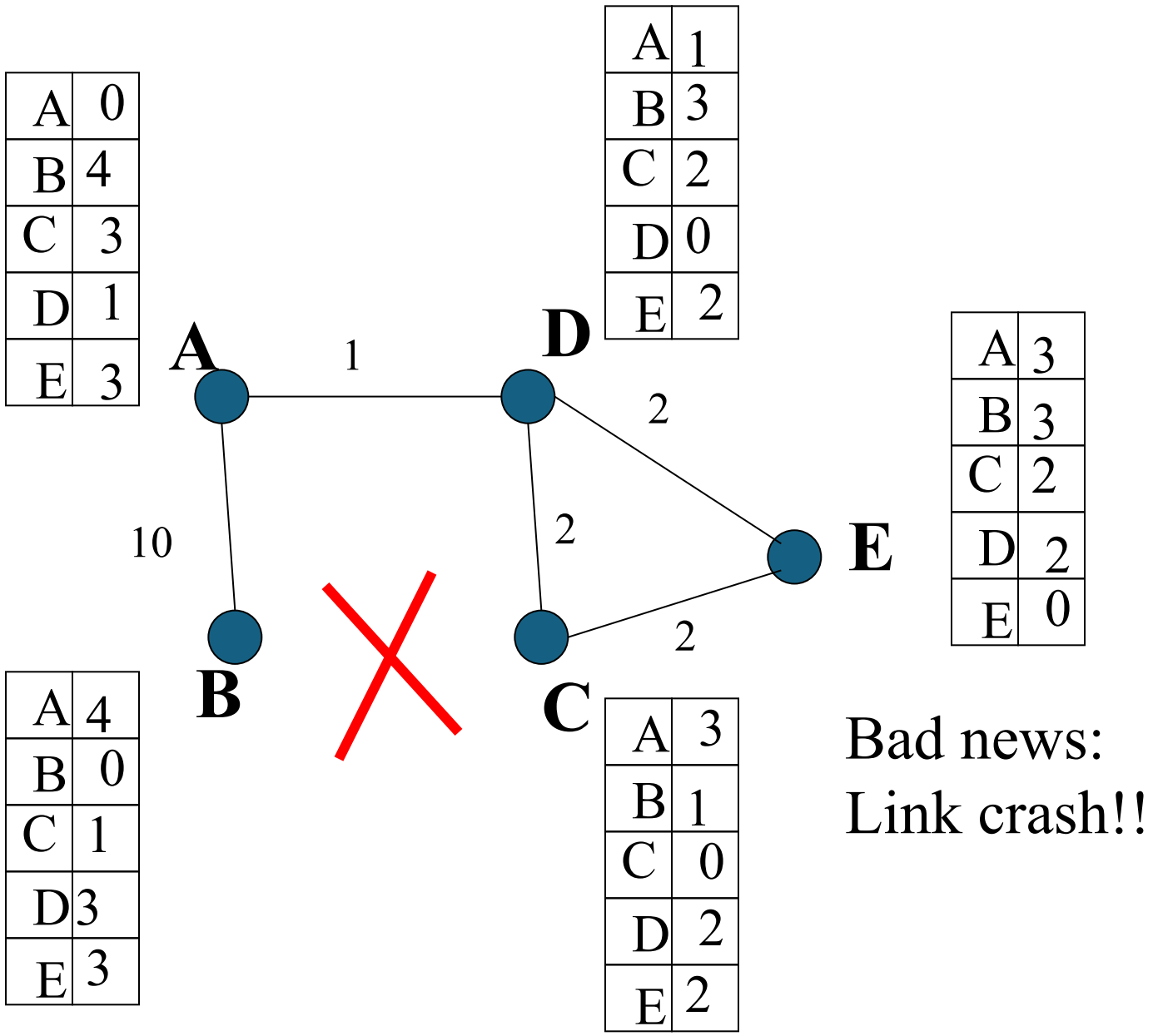

A
B
C
D
E
1
10
2
2
2
1
A 0
B 10
C 3
D 1
E 3
A 1
B 11
C 2
D 0
E 2
A 3
B 13
C 2
D 2
E 0
A 10
B 0
C 1
D 11
E 13
A 3
B 1
C 0
D 2
E 2
Direct endpoints know

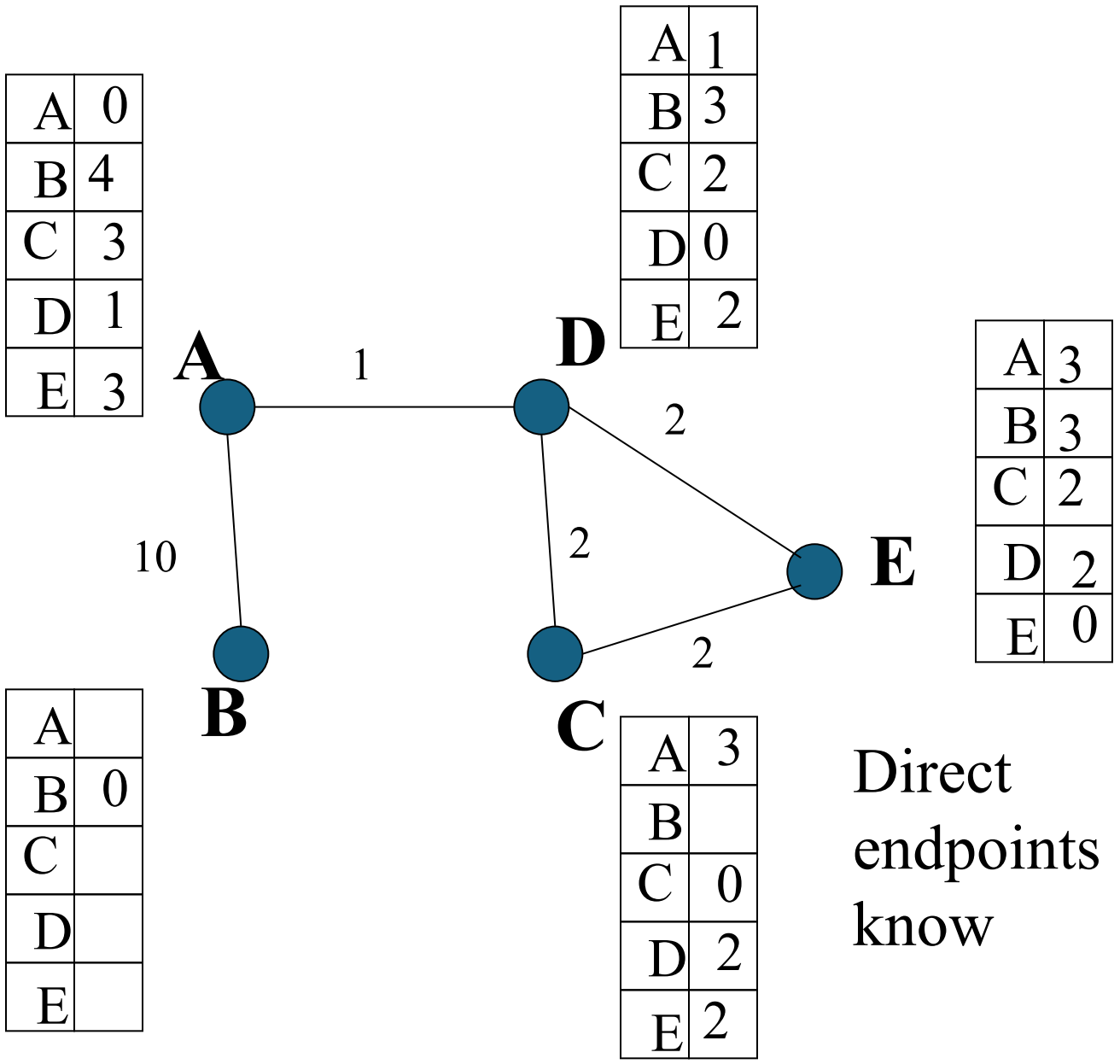

A
B
C
D
E
1
10
2
2
2
1
A 0
B 10
C 3
D 1
E 3
A 1
B 3
C 2
D 0
E 2
A 3
B 3
C 2
D 2
E 0
A 4
B 0
C 1
D 3
E 3
A 3
B 1
C 0
D 2
E 2
Neighbours know

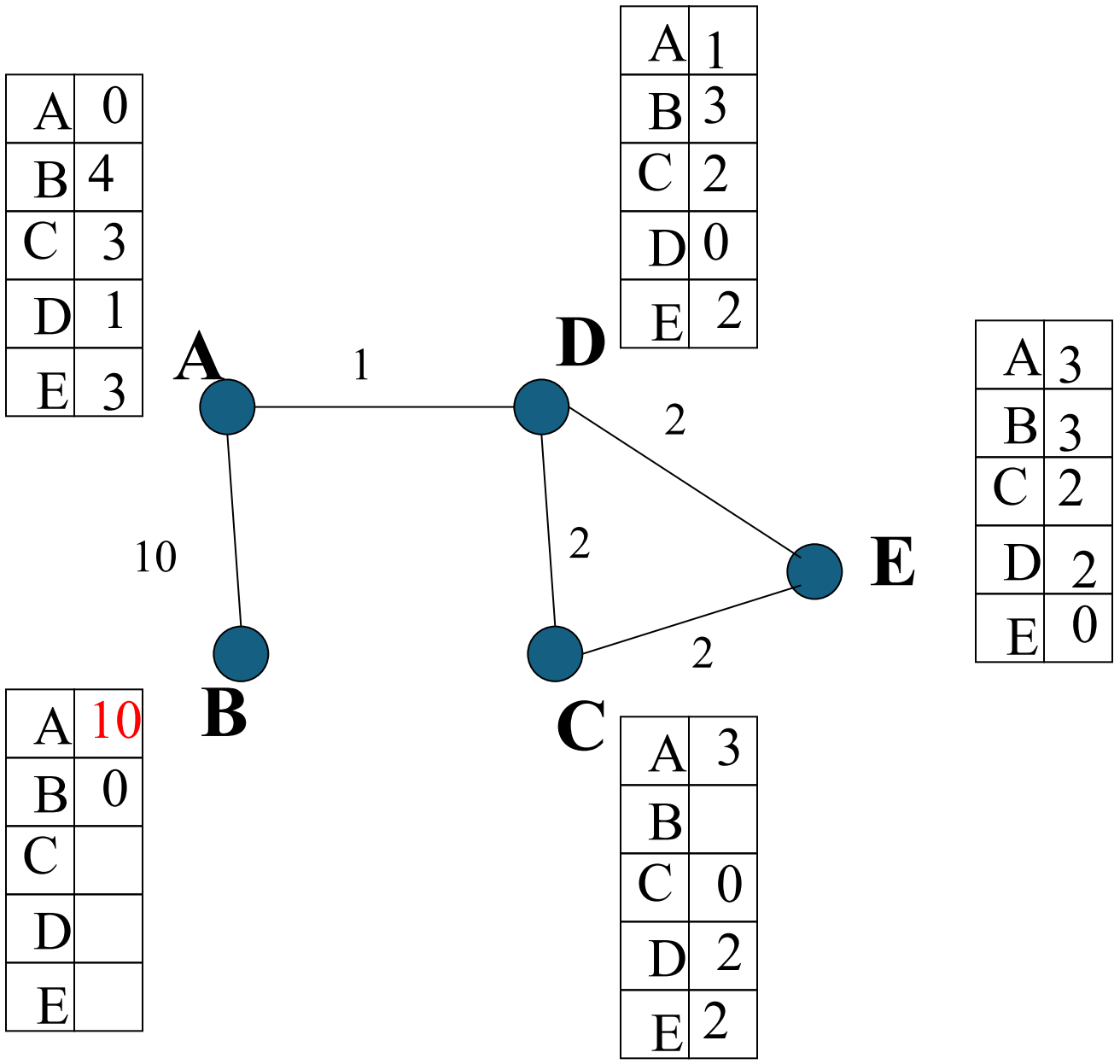

A
B
C
D
E
1
10
2
2
2
1
A 0
B 4
C 3
D 1
E 3
A 1
B 3
C 2
D 0
E 2
A 3
B 3
C 2
D 2
E 0
A 4
B 0
C 1
D 3
E 3
A 3
B 1
C 0
D 2
E 2
Neighbours of neighbours know

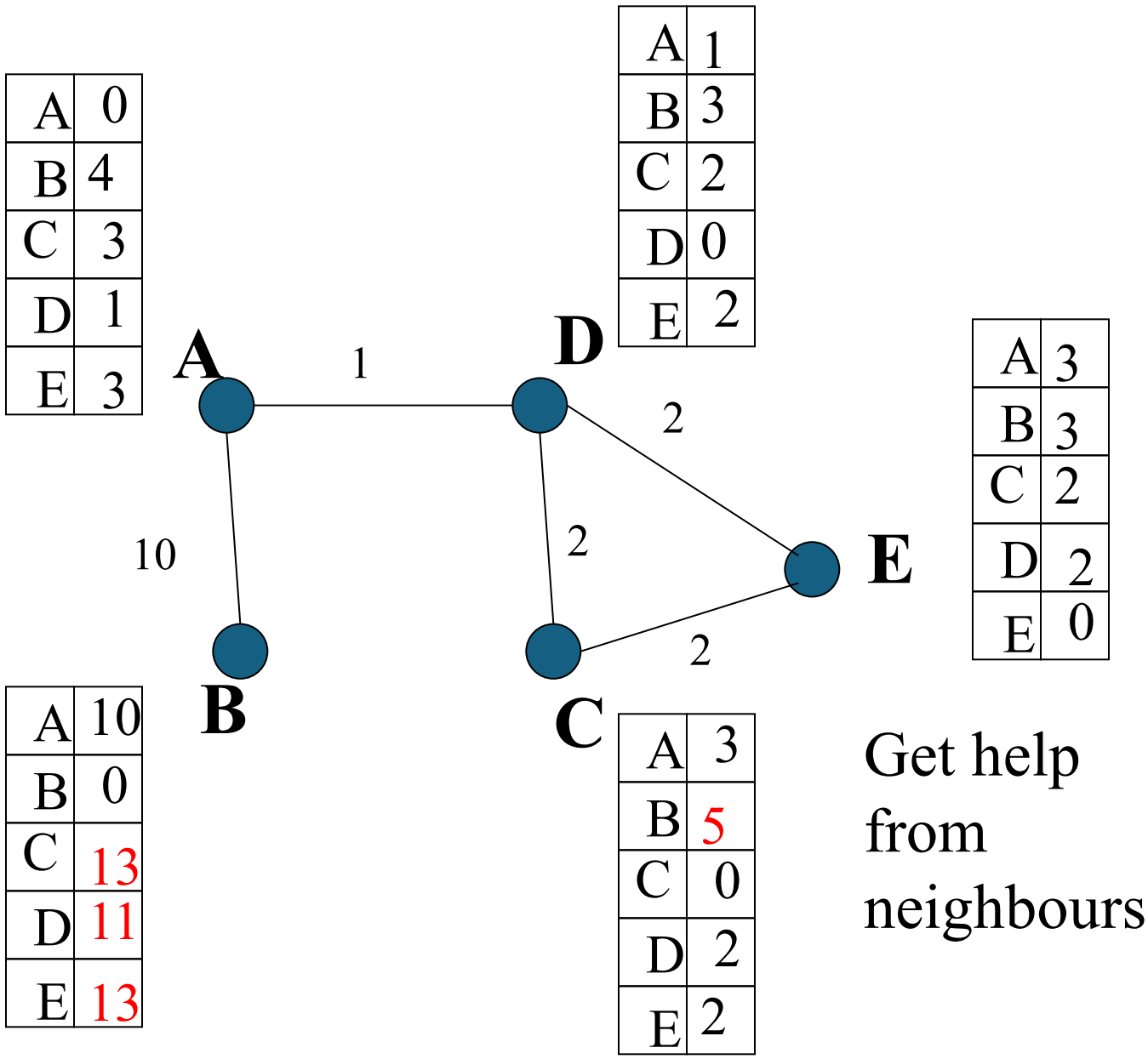


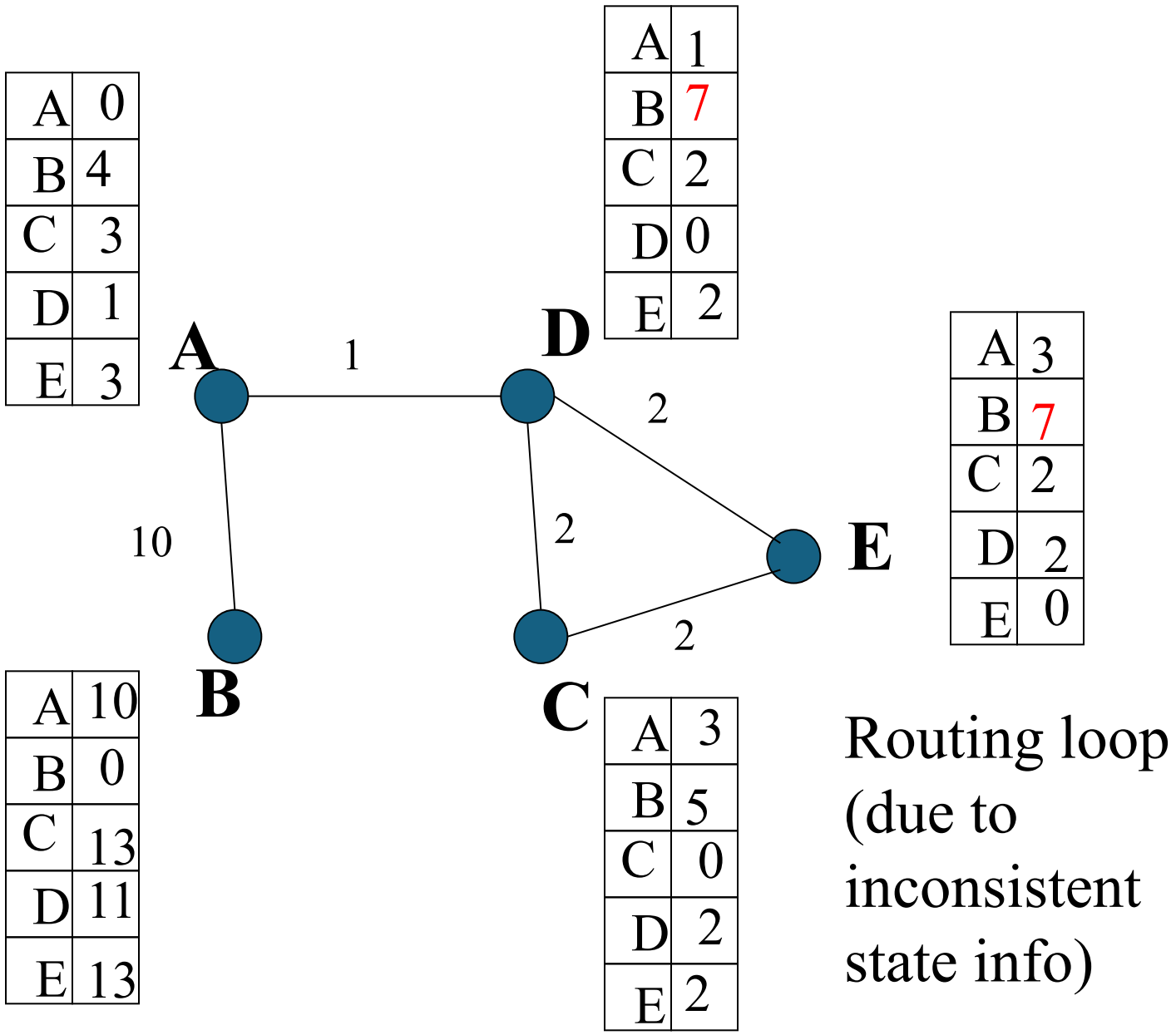

A happy and stable network


A
B
C
D
E
1
10
2
2
2
A 0
B 4
C 3
D 1
E 3
A 1
B 3
C 2
D 0
E 2
A 3
B 3
C 2
D 2
E 0
A 4
B 0
C 1
D 3
E 3
A 3
B 1
C 0
D 2
E 2
Bad news:
Link crash!!

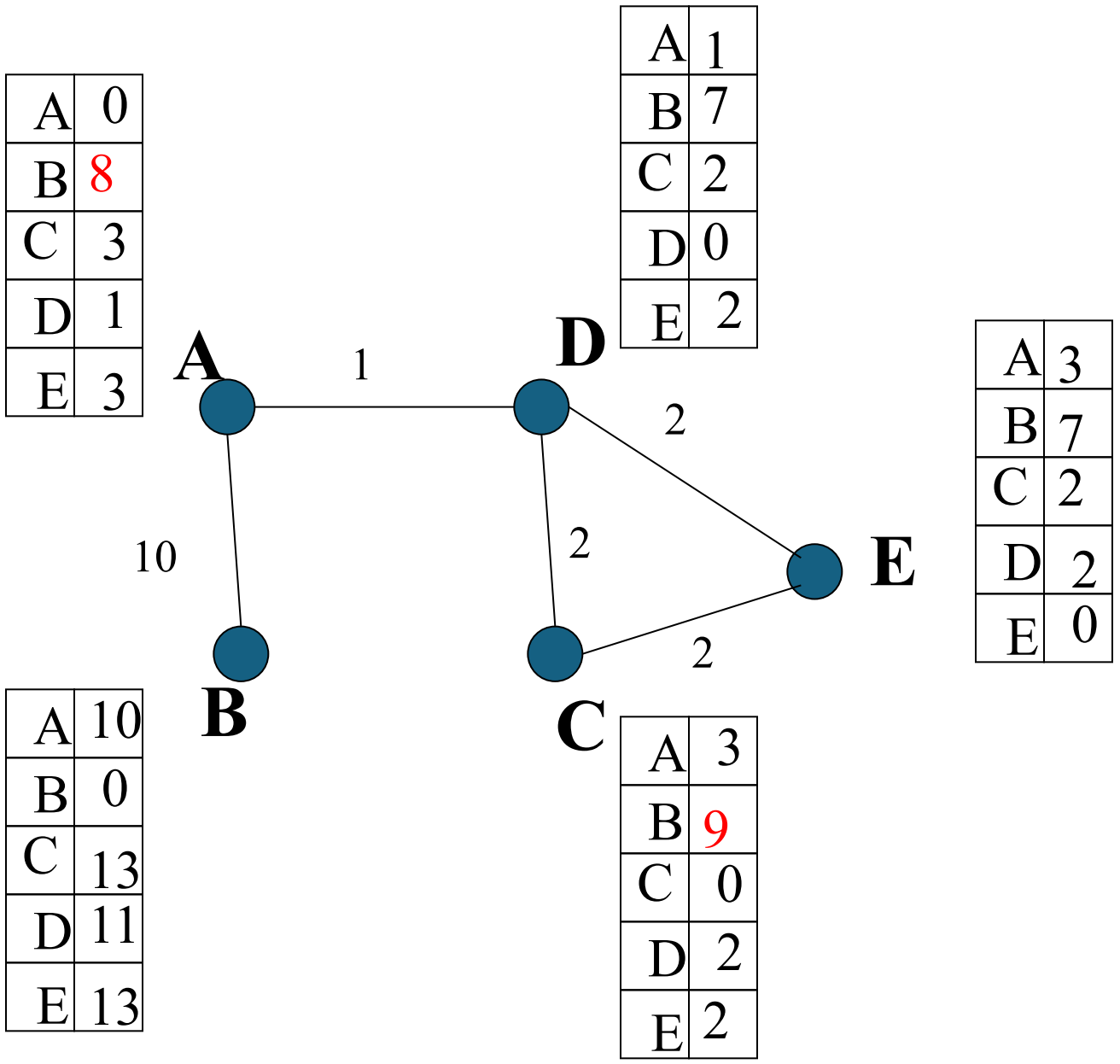

A
B
C
D
E
1
10
2
2
2
A 0
B 4
C 3
D 1
E 3
A 1
B 3
C 2
D 0
E 2
A 3
B 3
C 2
D 2
E 0
A
B 0
C
D
E
A 3
B
C 0
D 2
E 2
Direct endpoints know

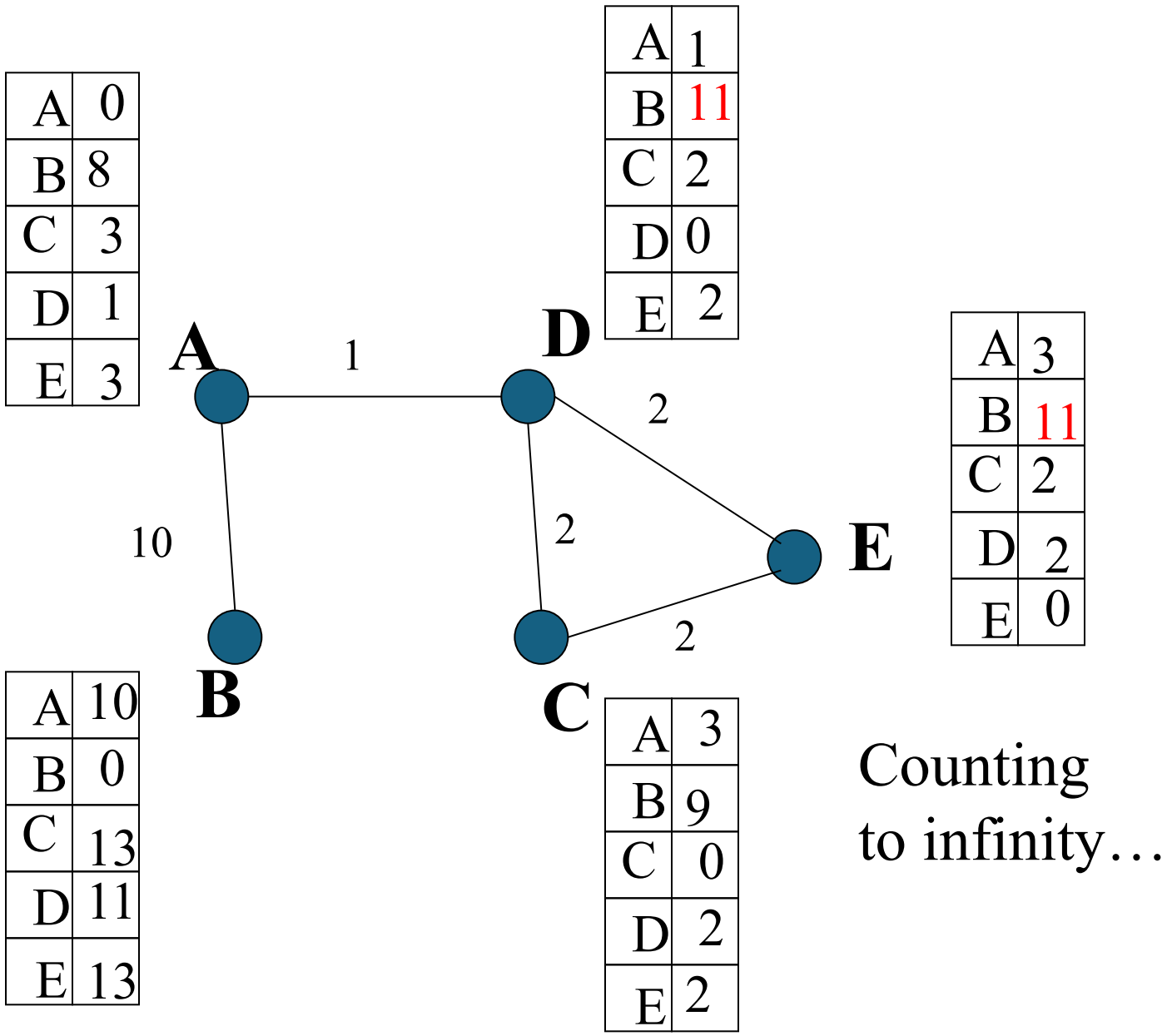

A
B
C
D
E
1
10
2
2
2
A 0
B 4
C 3
D 1
E 3
A 1
B 3
C 2
D 0
E 2
A 3
B 3
C 2
D 2
E 0
A 10
B 0
C
D
E
A 3
B
C 0
D 2
E 2

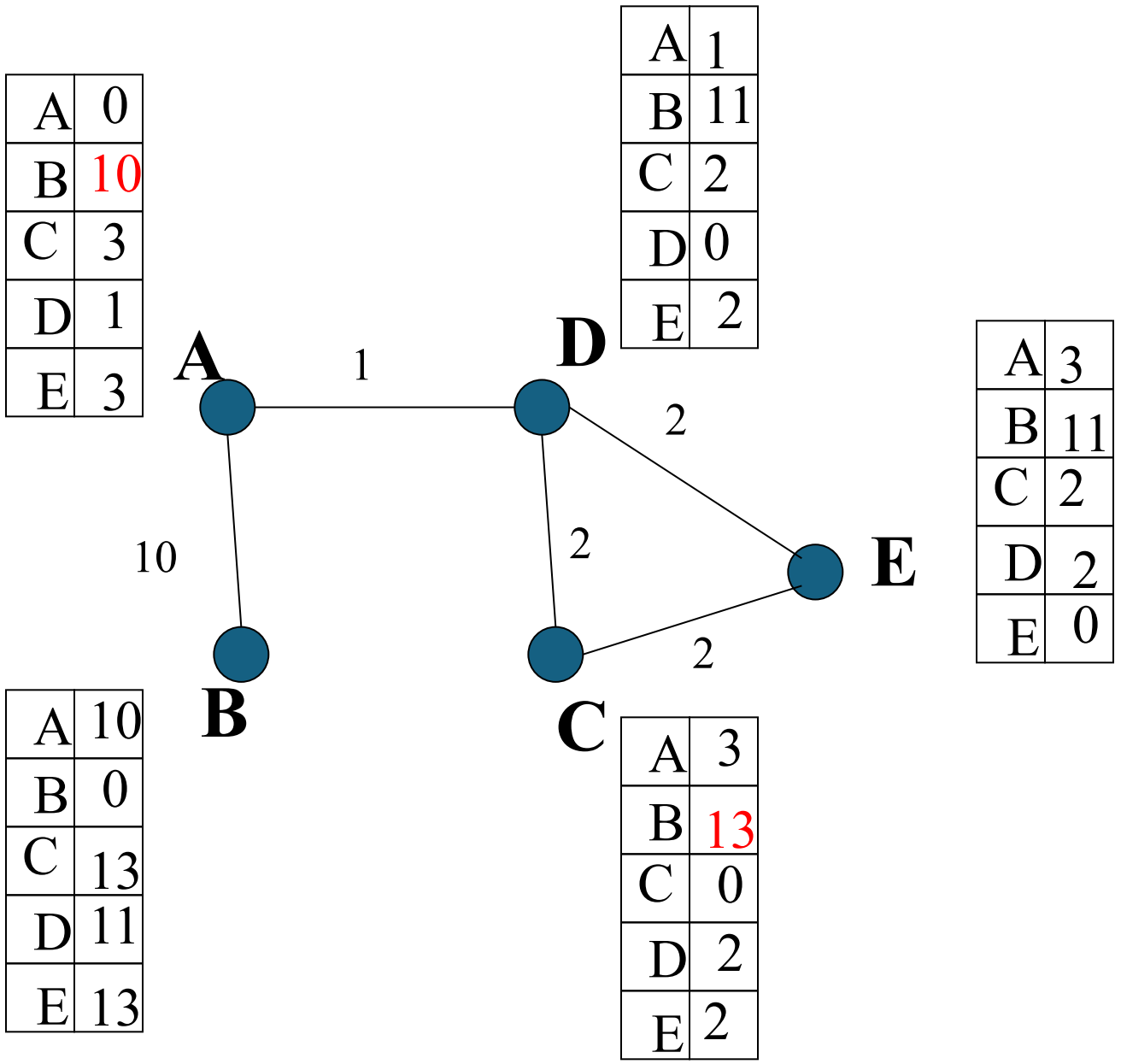

A
B
C
D
E
1
10
2
2
2
A 0
B 4
C 3
D 1
E 3
A 1
B 3
C 2
D 0
E 2
A 3
B 3
C 2
D 2
E 0
A 10
B 0
C 13
D 11
E 13
A 3
B 5
C 0
D 2
E 2
Get help from neighbours

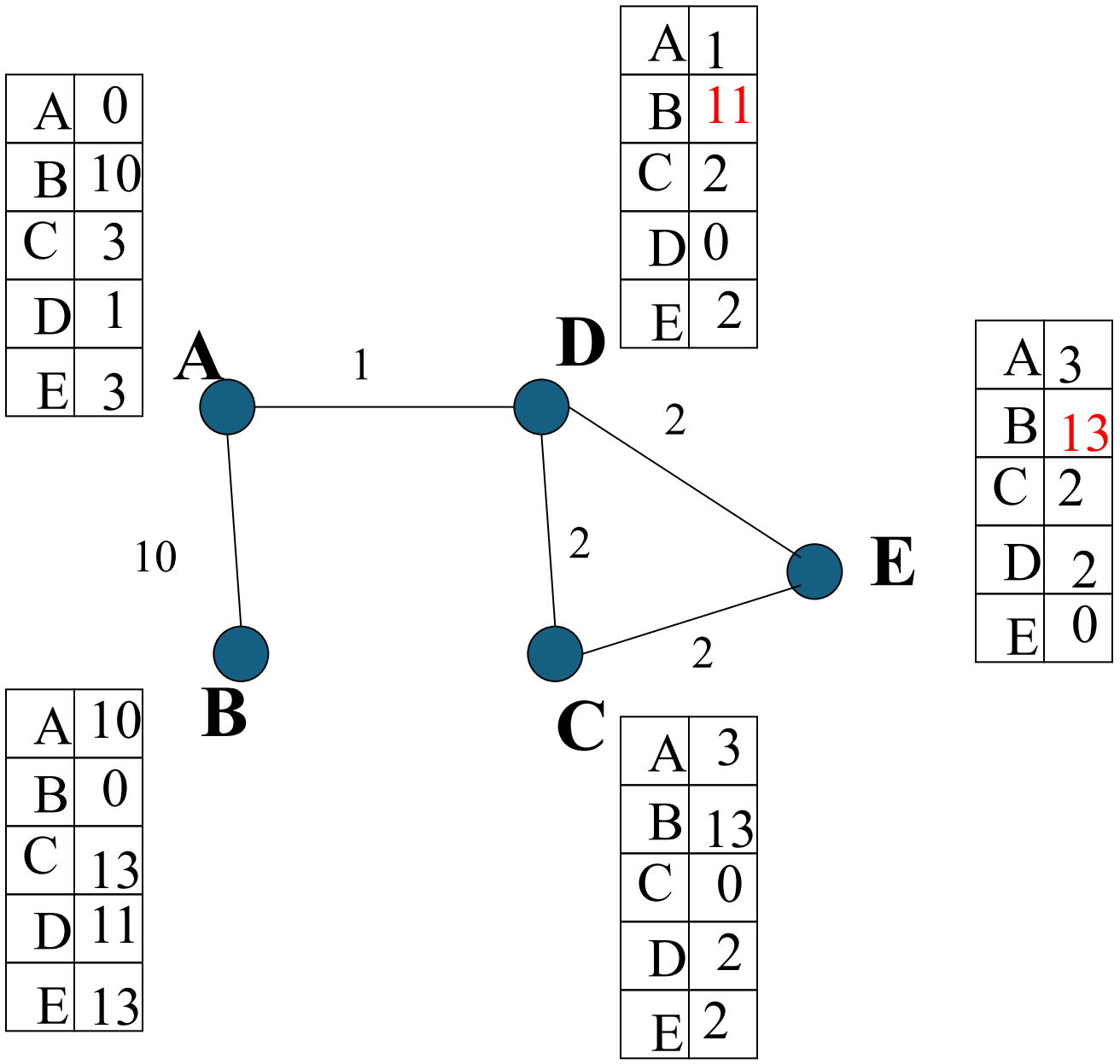


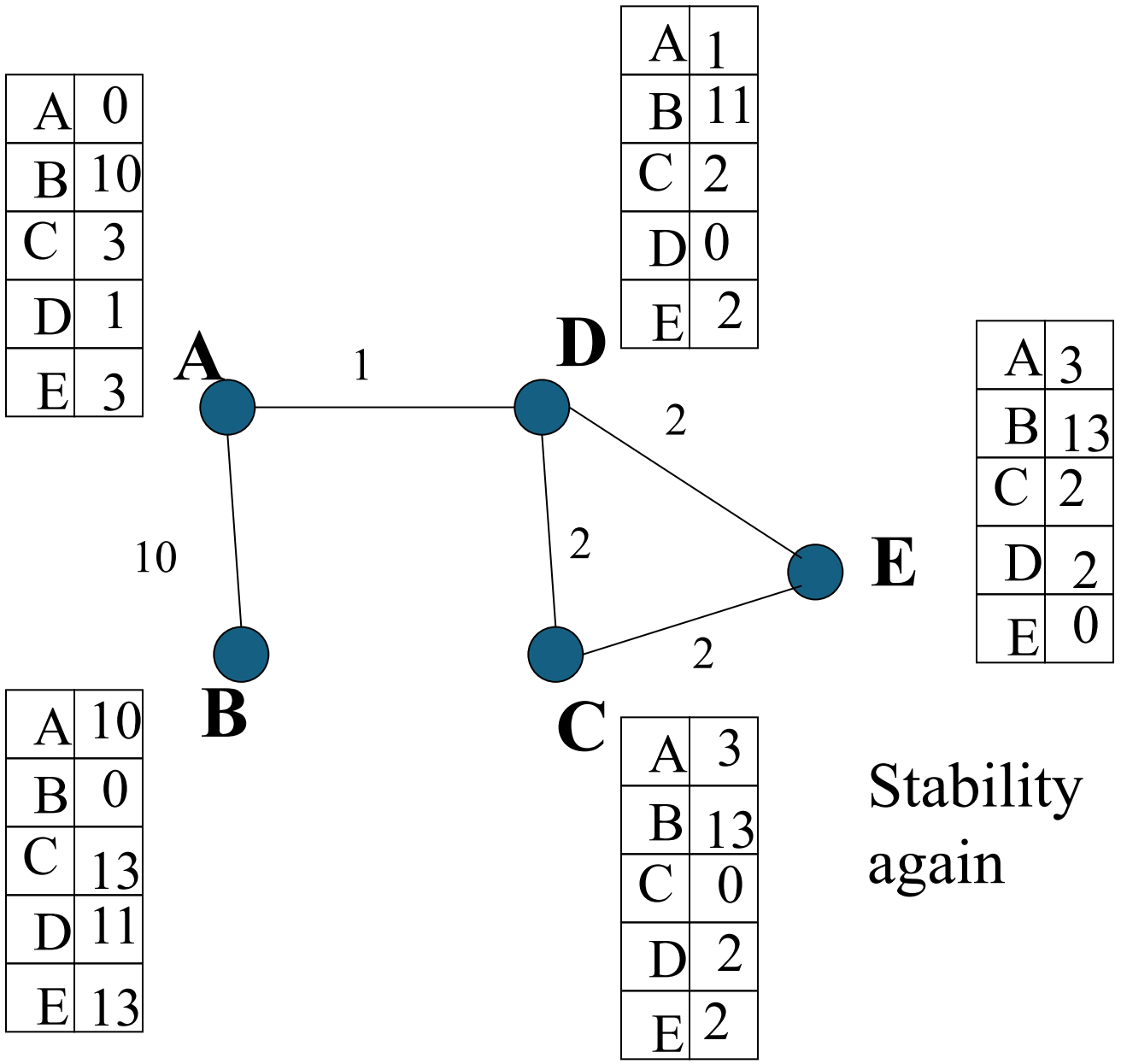

Routing loop (due to inconsistent state info)


A
B
C
D
E
1
10
2
2
2
A 0
B 8
C 3
D 1
E 3
A 1
B 7
C 2
D 0
E 2
A 3
B 7
C 2
D 2
E 0
A 10
B 0
C 13
D 11
E 13
A 3
B 9
C 0
D 2
E 2


A
B
C
D
E
1
10
2
2
2
A 0
B 8
C 3
D 1
E 3
A 1
B 11
C 2
D 0
E 2
A 3
B 11
C 2
D 2
E 0
A 10
B 0
C 13
D 11
E 13
A 3
B 9
C 0
D 2
E 2
Counting
to infinity…


A
B
C
D
E
1
10
2
2
2
A 0
B 10
C 3
D 1
E 3
A 1
B 11
C 2
D 0
E 2
A 3
B 11
C 2
D 2
E 0
A 10
B 0
C 13
D 11
E 13
A 3
B 13
C 0
D 2
E 2


A
B
C
D
E
1
10
2
2
2
A 0
B 10
C 3
D 1
E 3
A 1
B 11
C 2
D 0
E 2
A 3
B 13
C 2
D 2
E 0
A 10
B 0
C 13
D 11
E 13
A 3
B 13
C 0
D 2
E 2


A
B
C
D
E
1
10
2
2
2
A 0
B 10
C 3
D 1
E 3
A 1
B 11
C 2
D 0
E 2
A 3
B 13
C 2
D 2
E 0
A 10
B 0
C 13
D 11
E 13
A 3
B 13
C 0
D 2
E 2
Stability again

# Link State Routing

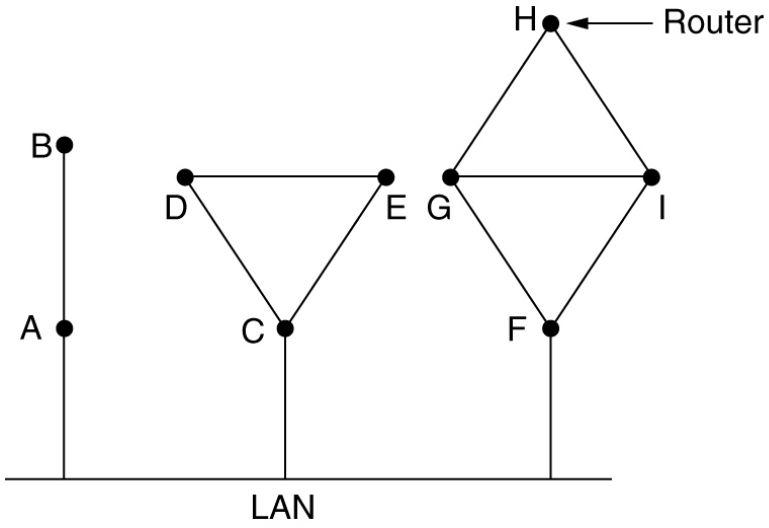
- OSPF (Open Shortest Path First)
- IS-IS (Intermediate System to Intermediate System)

Each router must do the following:

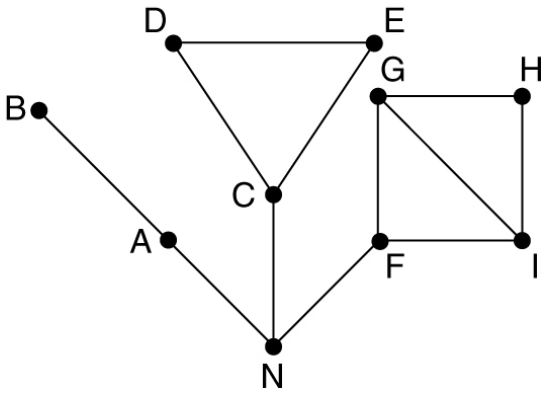
1. Discover its neighbors, learn their network address.
2. Measure the delay or cost to each of its neighbors.
3. Construct a packet telling all it has just learned.
4. Send this packet to all other routers.
5. Compute the shortest path to every other router.

# 1. Learning about the Neighbors

- **Hello packet** on each point to point line
- Routers have to respond with their name




(a)



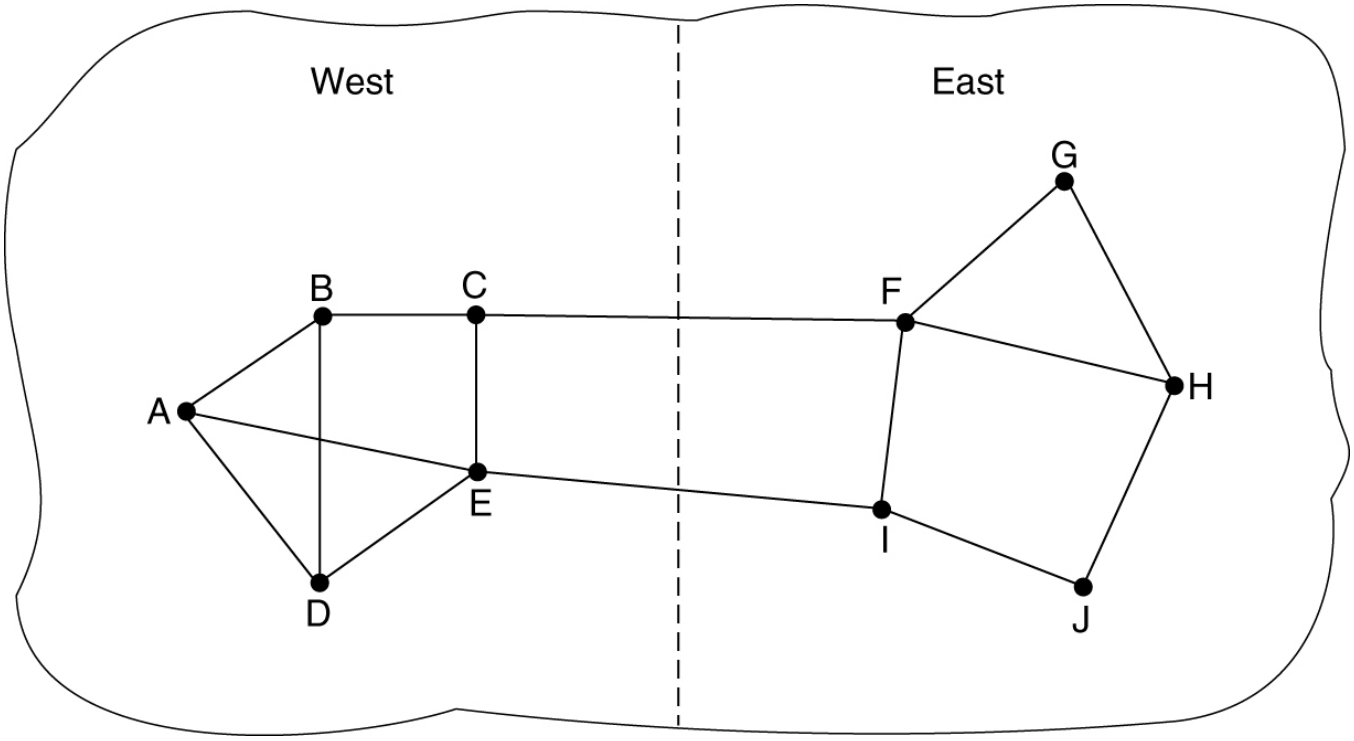

(b)

(a) Nine routers and a LAN.

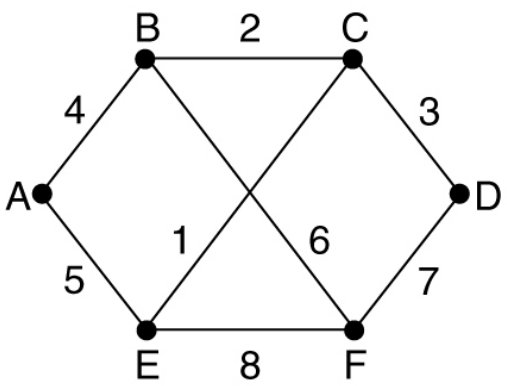
(b) A graph model of (a).

# 2. Measuring Line Cost

- A subnet in which the East and West parts are connected by two lines.



# 3 Building Link State Packets




(a)

Link State Packets

| A | |
|---|---|
| Seq. | |
| Age | |
| B | 4 |
| E | 5 |

| B | |
|---|---|
| Seq. | |
| Age | |
| A | 4 |
| C | 2 |
| F | 6 |

| C | |
|---|---|
| Seq. | |
| Age | |
| B | 2 |
| D | 3 |
| E | 1 |

| D | |
|---|---|
| Seq. | |
| Age | |
| C | 3 |
| F | 7 |

| E | |
|---|---|
| Seq. | |
| Age | |
| A | 5 |
| C | 1 |
| F | 8 |

| F | |
|---|---|
| Seq. | |
| Age | |
| B | 6 |
| D | 7 |
| E | 8 |

(b)

- (a) A subnet. (b) The link state packets for this subnet.

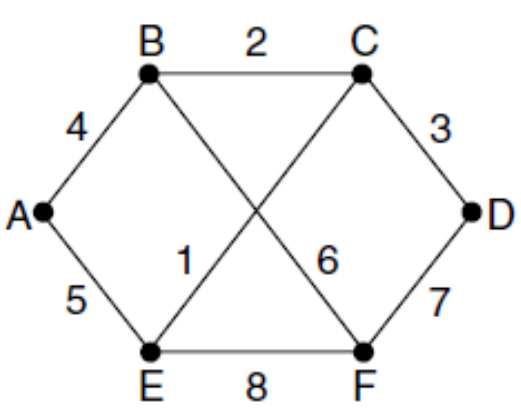
# 4 Link state packets

- Packet—When to Build

  - Identity of the sender
  - Seq Number
  - Age
  - List of Neighbours with Cost

When to build and send these packets

1. Build them periodically
2. When significant change in topology occurs




(a)

Link State Packets

| A | |
|---|---|
| Seq. | |
| Age | |
| B | 4 |
| E | 5 |

| B | |
|---|---|
| Seq. | |
| Age | |
| A | 4 |
| C | 2 |
| F | 6 |

| C | |
|---|---|
| Seq. | |
| Age | |
| B | 2 |
| D | 3 |
| E | 1 |

| D | |
|---|---|
| Seq. | |
| Age | |
| C | 3 |
| F | 7 |

| E | |
|---|---|
| Seq. | |
| Age | |
| A | 5 |
| C | 1 |
| F | 8 |

| F | |
|---|---|
| Seq. | |
| Age | |
| B | 6 |
| D | 7 |
| E | 8 |

(b)

**Figure 5-12.** (a) A network. (b) The link state packets for this network.

# Link State Routing : Distributing the Link State Packets

**Basic Idea:**

- Flooding
- Router keeps track of Source Router, Seq Number pair to prevent congestion.
- If new forward, if already in list discard
- Lower sequence number rejected

**Possible Issues**

- Sequence number Warps around? –confusion
- Solution: 32 bit sequence number –one link state packet per second, how long will it sustain?
- If a router crashes??
- IF seq number part of the packet is corrupted?

Age

# Distributing the Link State Packets

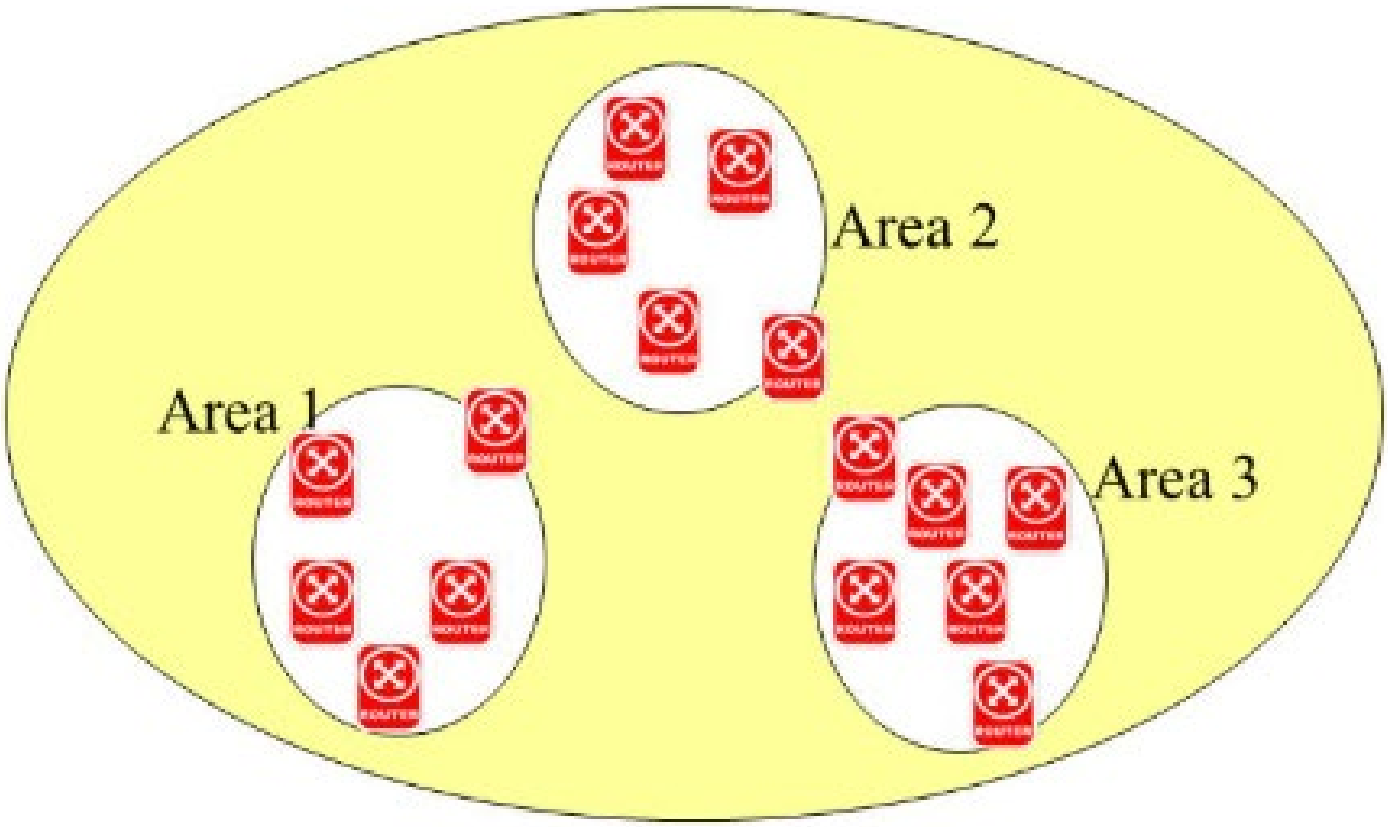
- The packet buffer for router B in the previous slide (Fig. 5-13).

| | | | Send flags | | | ACK flags | | | |
|---|---|---|---|---|---|---|---|---|---|
| Source | Seq. | Age | A | C | F | A | C | F | Data |
| A | 21 | 60 | 0 | 1 | 1 | 1 | 0 | 0 | |
| F | 21 | 60 | 1 | 1 | 0 | 0 | 0 | 1 | |
| E | 21 | 59 | 0 | 1 | 0 | 1 | 0 | 1 | |
| C | 20 | 60 | 1 | 0 | 1 | 0 | 1 | 0 | |
| D | 21 | 59 | 1 | 0 | 0 | 0 | 1 | 1 | |

# Hierarchical Routing

- As networks grow in size, the router routing tables grow proportionally.
- Router memory consumed by ever-increasing tables,
- More CPU time is needed to scan them
- More bandwidth is needed to send status reports about them.
- At a certain point, the network may grow to the point where it is no longer feasible for every router to have an entry for every other router,
- Routing will have to be done hierarchically,
  - as it is in the telephone network
- routers are divided into **regions**.
- Each router knows all the details about how to route packets to destinations within its own region

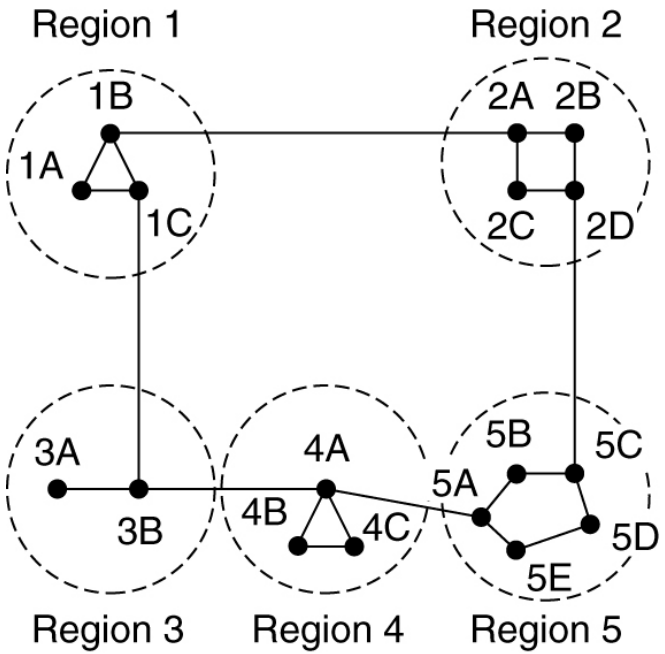
# Hierarchical Routing



# Hierarchical Routing

- For huge networks, a two-level hierarchy may be insufficient;
- it may be necessary to group
  - the regions into clusters,
  - the clusters into zones,
  - the zones into groups,
  - and so on,

# Hierarchical Routing




(a)

Full table for 1A

| Dest. | Line | Hops |
|---|---|---|
| 1A | – | – |
| 1B | 1B | 1 |
| 1C | 1C | 1 |
| 2A | 1B | 2 |
| 2B | 1B | 3 |
| 2C | 1B | 3 |
| 2D | 1B | 4 |
| 3A | 1C | 3 |
| 3B | 1C | 2 |
| 4A | 1C | 3 |
| 4B | 1C | 4 |
| 4C | 1C | 4 |
| 5A | 1C | 4 |
| 5B | 1C | 5 |
| 5C | 1B | 5 |
| 5D | 1C | 6 |
| 5E | 1C | 5 |

(b)

Hierarchical table for 1A

| Dest. | Line | Hops |
|---|---|---|
| 1A | – | – |
| 1B | 1B | 1 |
| 1C | 1C | 1 |
| 2 | 1B | 2 |
| 3 | 1C | 2 |
| 4 | 1C | 3 |
| 5 | 1C | 4 |

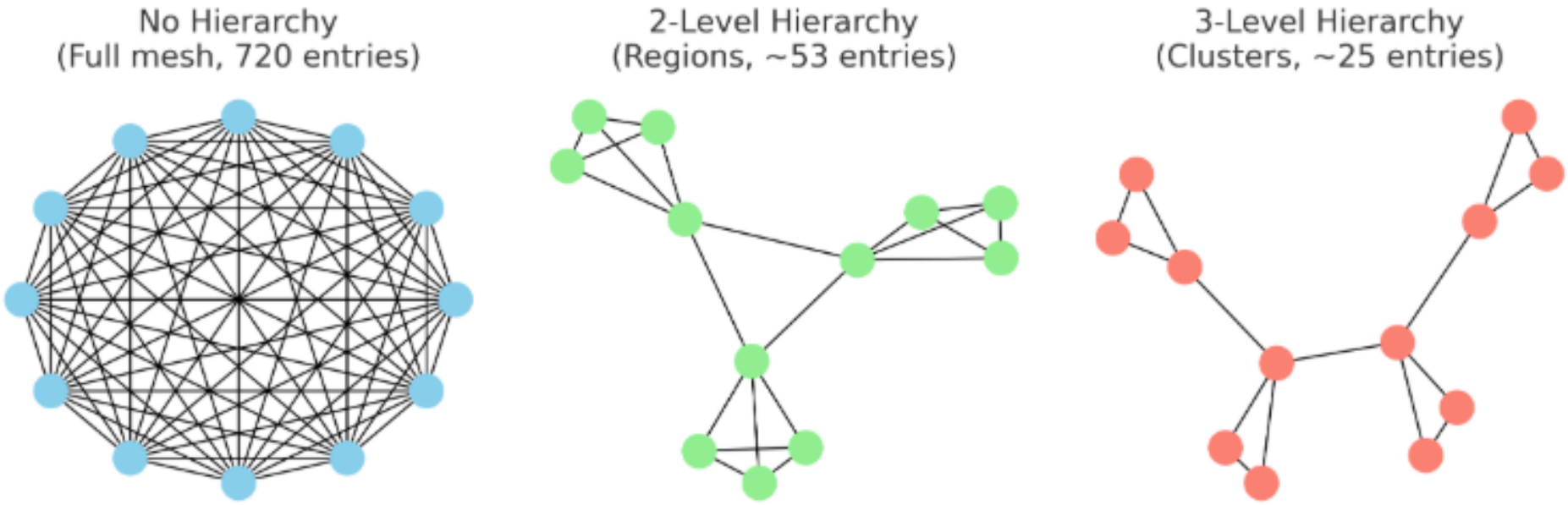
(c)

# Hierarchical Routing

- There is a penalty to be paid.
- Increased path length.

- **Hierarchical Routing – Optimal Levels**
- **Problem:** Large networks → routing tables grow rapidly.
- **Example (720 routers):**
  - **No hierarchy:** 720 entries per router.
  - **2-level hierarchy (24 regions × 30):** 53 entries per router.
  - **3-level hierarchy (8 clusters × 9 regions × 10):** 25 entries per router.
- **Result (Kamoun & Kleinrock, 1979):**
  - Optimal levels ≈ **ln(N)**
  - Routing table size ≈ **e ln(N)** entries per router.
  - Path length increase = **small & acceptable**.

# Hierarchical Routing




- **No hierarchy** → every router talks to every other.
- **2-level hierarchy** → regions reduce entries.
- **3-level hierarchy** → clusters + regions minimize entries further.

# Broadcast Routing

- Sending a packet from one source to **all other nodes** in the network.

- **Methods:**

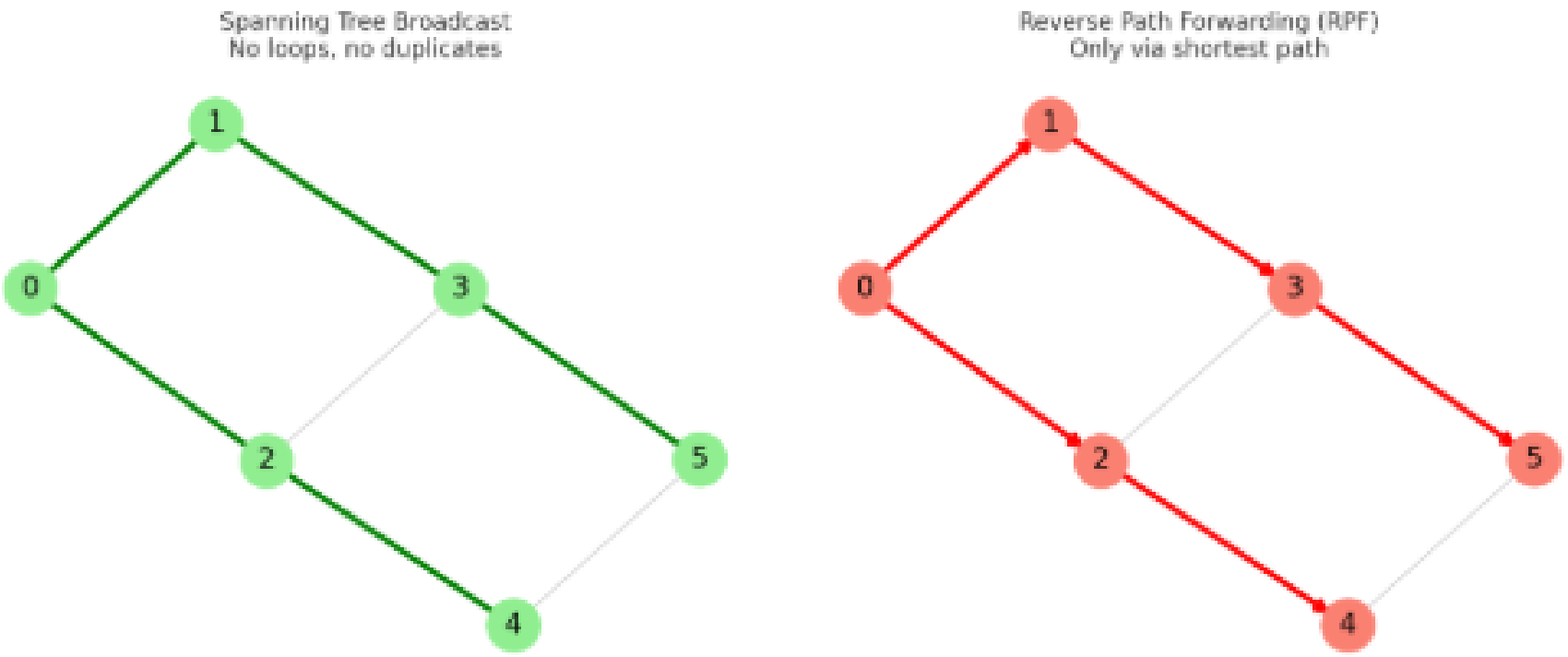
- **Flooding**
  - Each incoming packet is sent out on *all outgoing links* (except the one it arrived on).
  - Ensures delivery but may create duplicates.
  - **Solution:** Sequence numbers used to detect and discard duplicates.
  - Simple but inefficient.

- **Multi-Destination Routing**
  - Packet header contains a **list of destinations**.
  - At each router, the list is split according to outgoing paths.
  - Reduces duplicates but requires larger headers.
  - Bandwidth Utilization is effective

# Broadcast Routing

- **Spanning Tree Broadcast**
  - Construct a **spanning tree** of the network.
  - Packets are forwarded along tree branches only → **no loops, no duplicates**.
  - Efficient, commonly used.
- **Reverse Path Forwarding (RPF)**
  - Router forwards a broadcast packet *only if it arrived on the shortest path* from source.
  - Prevents loops and excess transmissions.
- **Applications:**
- Routing table updates (distance vector, link state)
- Service advertisements (e.g., ARP, DHCP)
- Distributed systems communication

## Broadcast Routing Methods



# Multicast Routing

- **Definition:**
- Transmission of a packet from **one source to a specific group of destinations**.
- More efficient than sending multiple unicasts or broadcasting to all.
- **Approaches:**
- **Flooding + Pruning**
  - Initially flood packets like broadcast.
  - Routers not part of multicast group send **prune messages** upstream.
  - Creates a **multicast tree** only for group members.
- **Reverse Path Forwarding (RPF) with Pruning**
  - Similar to broadcast RPF but only keeps branches leading to group members.
  - Prevents unnecessary forwarding.

# Multicast Routing

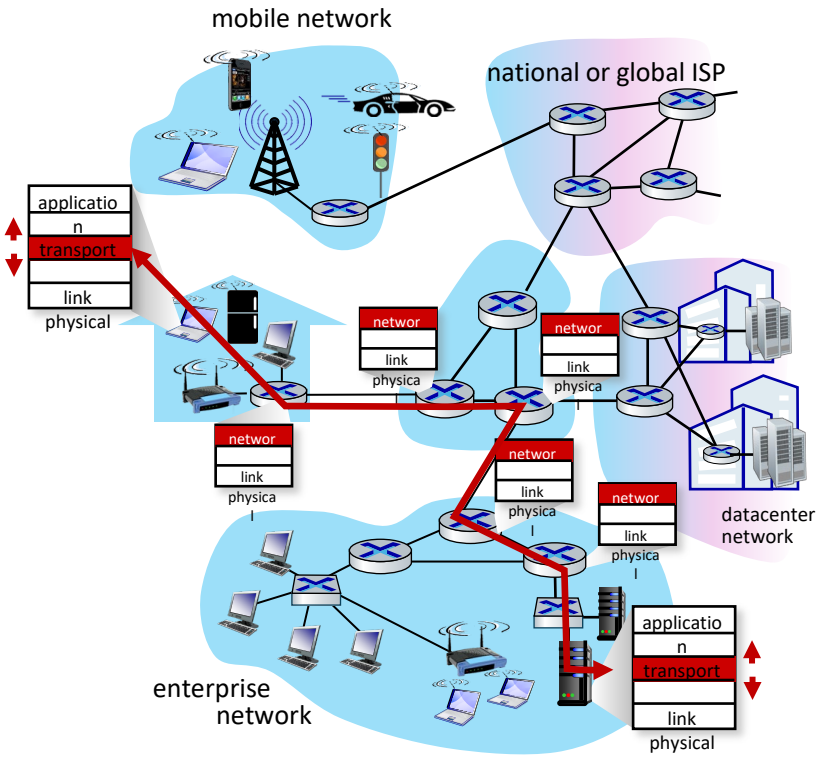
- **Core-Based Trees (CBT)**
  - Define a central router (“core” or “rendezvous point”).
  - Each member joins by sending a message toward the core.
  - Forms a **shared tree** for the group.
  - Less overhead than source-based trees.
- **Protocol Independent Multicast (PIM)**
  - Works in two modes:
    - **Dense Mode (PIM-DM):** Assumes many members, uses flood-and-prune.
    - **Sparse Mode (PIM-SM):** Assumes few members, uses core-based trees.

**Group Management:** Hosts join/leave groups using IGMP (Internet Group Management Protocol).
**Efficiency:** Delivers to only interested receivers.
**Applications:**

Video conferencing
Live streaming
Distributed systems updates

# Network-layer services and protocols

- transport segment from sending to receiving host
  - sender: encapsulates segments into datagrams, passes to link layer
  - receiver: delivers segments to transport layer protocol
- network layer protocols in *every Internet device*: hosts, routers
- routers:
  - examines header fields in all IP datagrams passing through it
  - moves datagrams from input ports to output ports to transfer datagrams along end-end path



# Two key network-layer functions

## network-layer functions:

- *forwarding:* move packets from a router's input link to appropriate router output link
  - *routing:* determine route taken by packets from source to destination
    - *routing algorithms*

## analogy: taking a trip

- *forwarding:* process of getting through single interchange
- *routing:* process of planning trip from source to destination



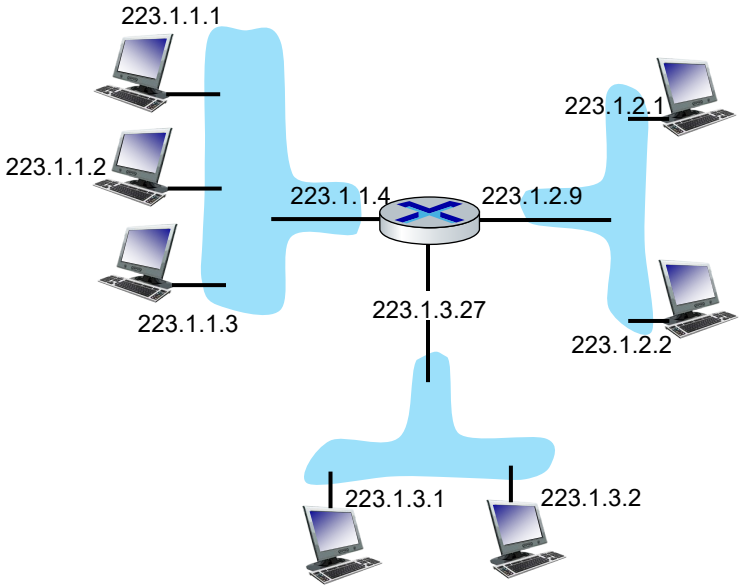
forwarding



routing

# IP addressing: introduction

- IP address: 32-bit identifier associated with each host or router *interface*
- interface: connection between host/router and physical link
  - router's typically have multiple interfaces
  - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)

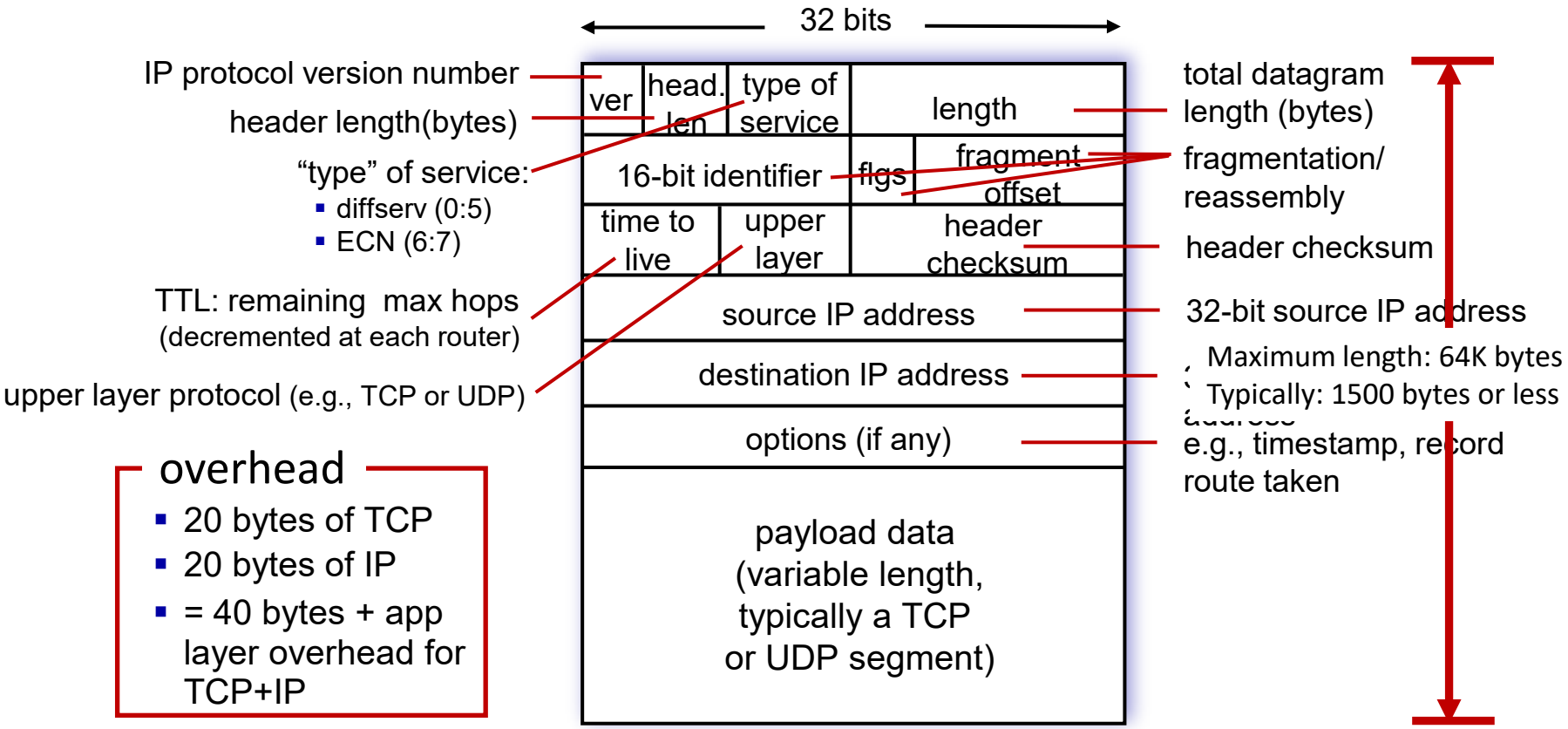



dotted-decimal IP address notation:

223.1.1.1 = 11011111 00000001 00000001 00000001

223 1 1 1

# IP Datagram format




## overhead

- 20 bytes of TCP
- 20 bytes of IP
- = 40 bytes + app layer overhead for TCP+IP

# DHCP: Dynamic Host Configuration Protocol

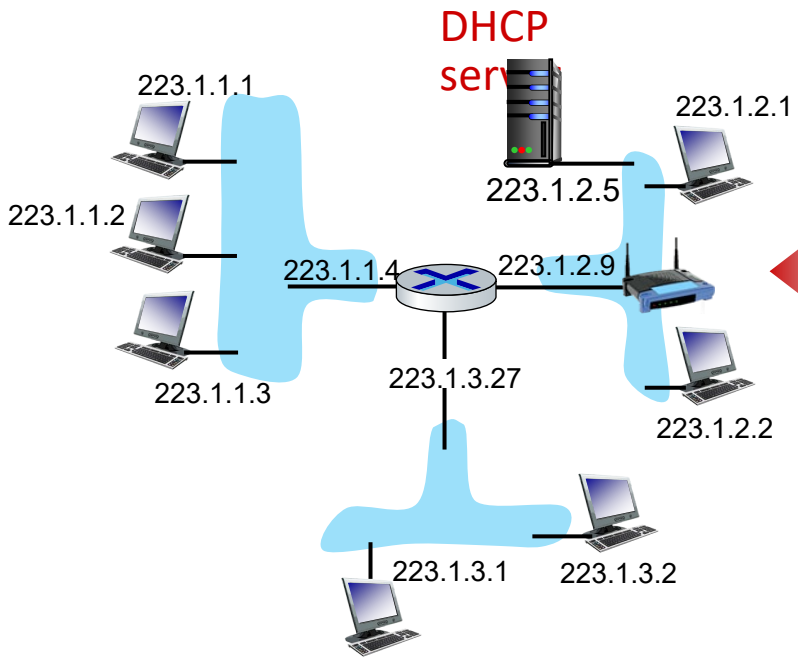
goal: host *dynamically* obtains IP address from network server when it "joins" network

- can renew its lease on address in use
- allows reuse of addresses (only hold address while connected/on)
- support for mobile users who join/leave network

DHCP overview:

- host broadcasts DHCP discover msg [optional]
- DHCP server responds with DHCP offer msg [optional]
- host requests IP address: DHCP request msg
- DHCP server sends address: DHCP ack msg

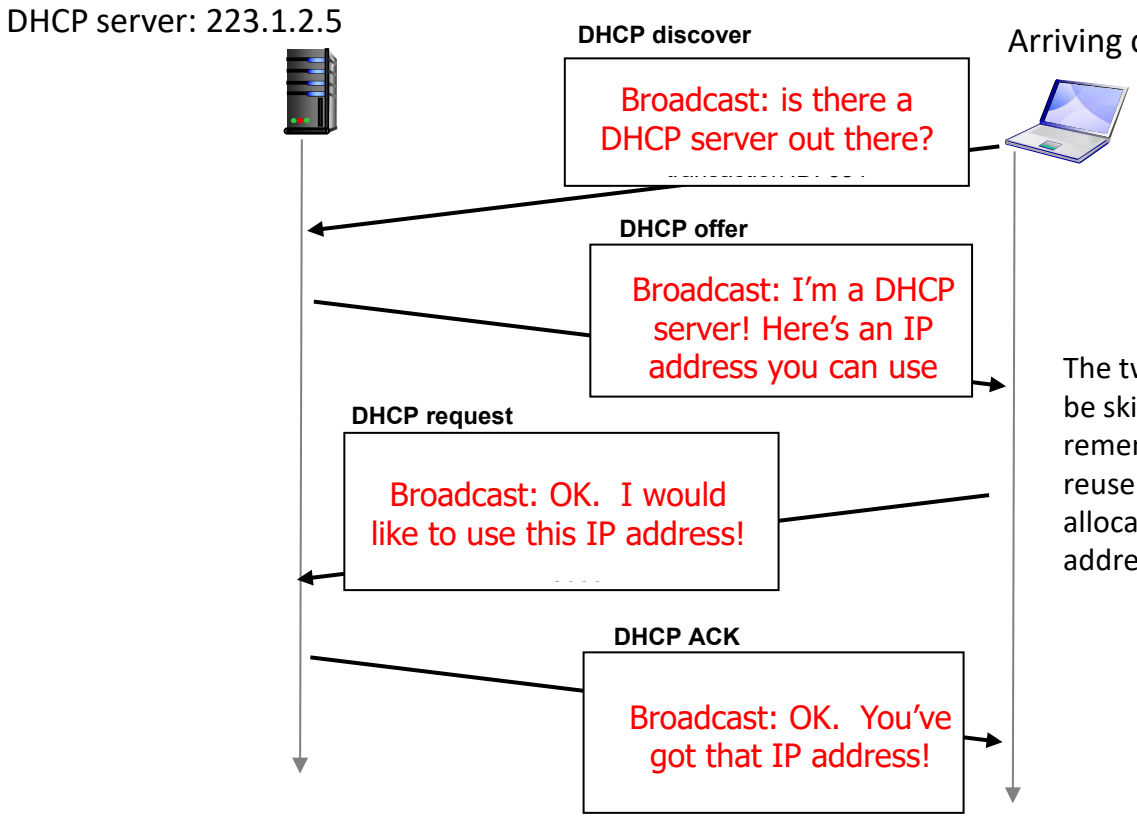
# DHCP client-server scenario




Typically, DHCP server will be co-located in router, serving all subnets to which router is attached

arriving DHCP client needs address in this network

# DHCP client-server scenario



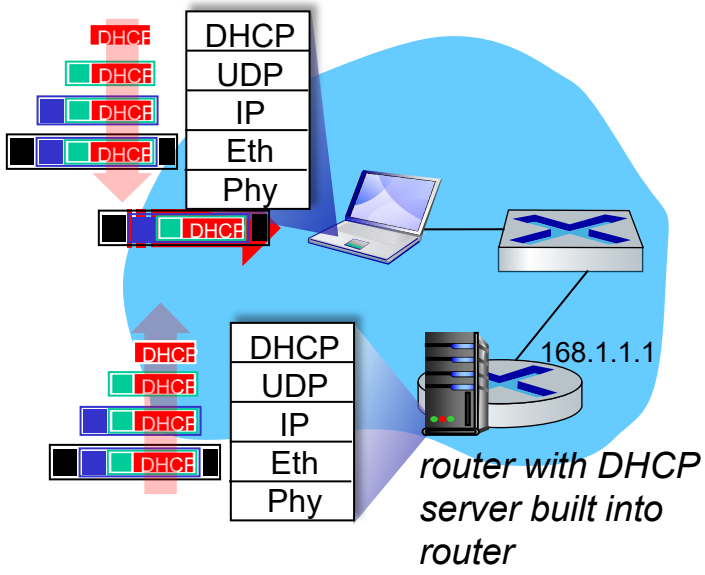

The two steps above can be skipped "if a client remembers and wishes to reuse a previously allocated network address" [RFC 2131]

# DHCP: more than IP addresses

DHCP can return more than just allocated IP address on subnet:

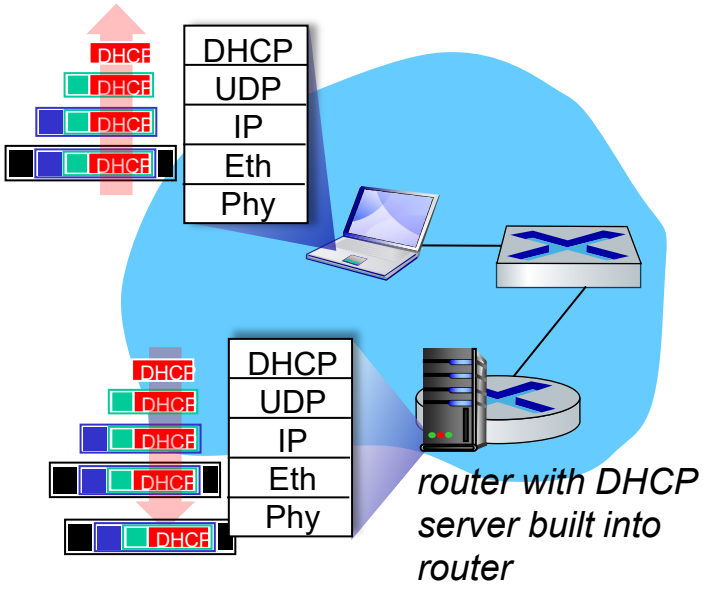
- address of first-hop router for client
- name and IP address of DNS sever
- network mask (indicating network versus host portion of address)

# DHCP: example




- Connecting laptop will use DHCP to get IP address, address of first-hop router, address of DNS server.
- DHCP REQUEST message encapsulated in UDP, encapsulated in IP, encapsulated in Ethernet
- Ethernet frame broadcast (dest: FFFFFFFFFFFF) on LAN, received at router running DHCP server
- Ethernet de-mux'ed to IP de-mux'ed, UDP de-mux'ed to DHCP

# DHCP: example

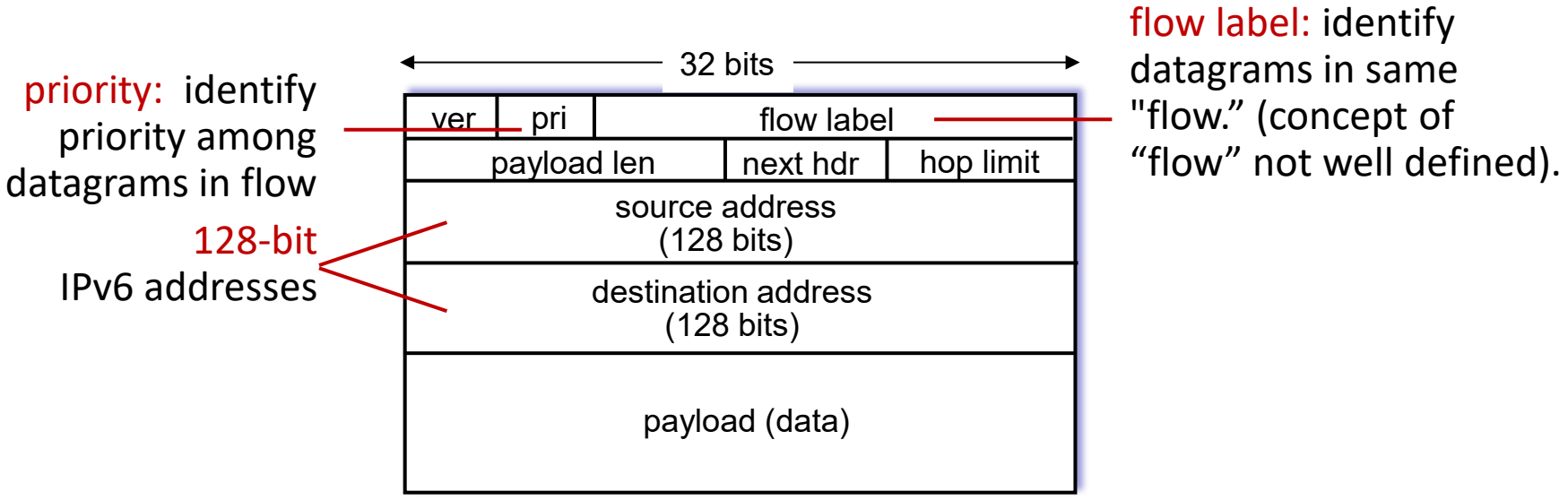



- DCP server formulates DHCP ACK containing client's IP address, IP address of first-hop router for client, name & IP address of DNS server
- encapsulated DHCP server reply forwarded to client, de-muxing up to DHCP at client
- client now knows its IP address, name and IP address of DNS server, IP address of its first-hop router

# IPv6: motivation

- initial motivation: 32-bit IPv4 address space would be completely allocated
- additional motivation:
  - speed processing/forwarding: 40-byte fixed length header
  - enable different network-layer treatment of “flows”

# IPv6 datagram format

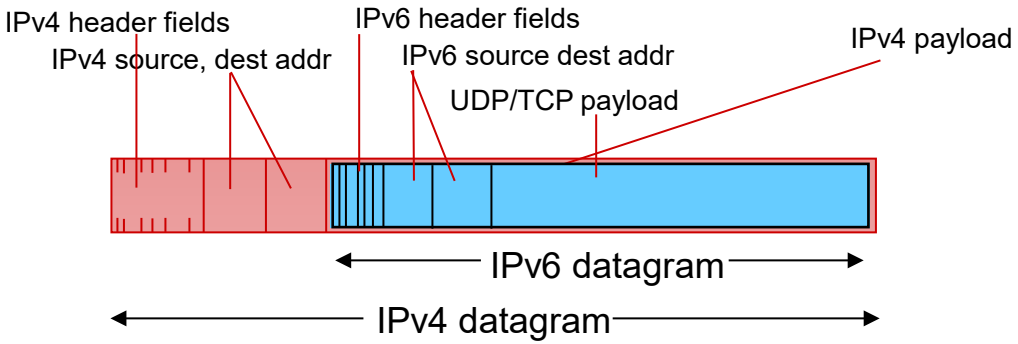



What's missing (compared with IPv4):

- no checksum (to speed processing at routers)
- no fragmentation/reassembly
- no options (available as upper-layer, next-header protocol at router)

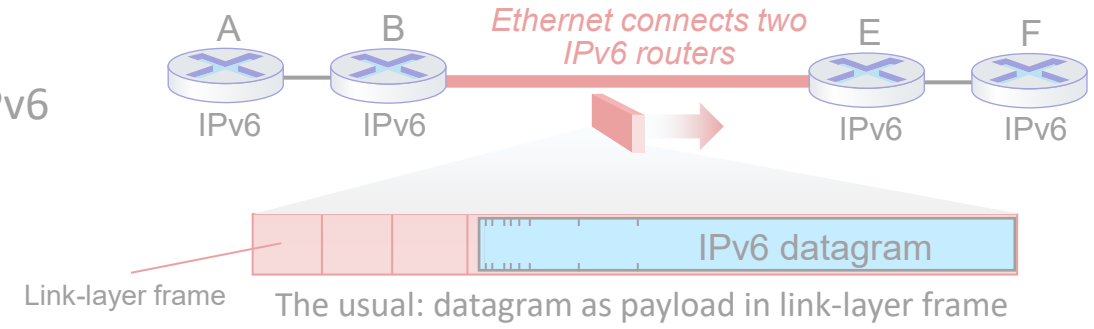
# Transition from IPv4 to IPv6

- not all routers can be upgraded simultaneously
  - no “flag days”
  - how will network operate with mixed IPv4 and IPv6 routers?
- tunneling: IPv6 datagram carried as *payload* in IPv4 datagram among IPv4 routers (“packet within a packet”)
  - tunneling used extensively in other contexts (4G/5G)

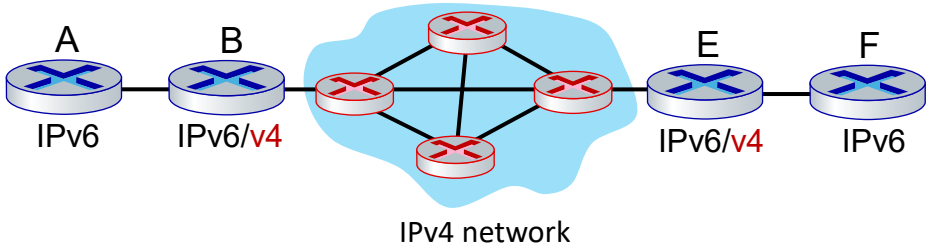


# Tunneling and encapsulation

Ethernet connecting two IPv6 routers:

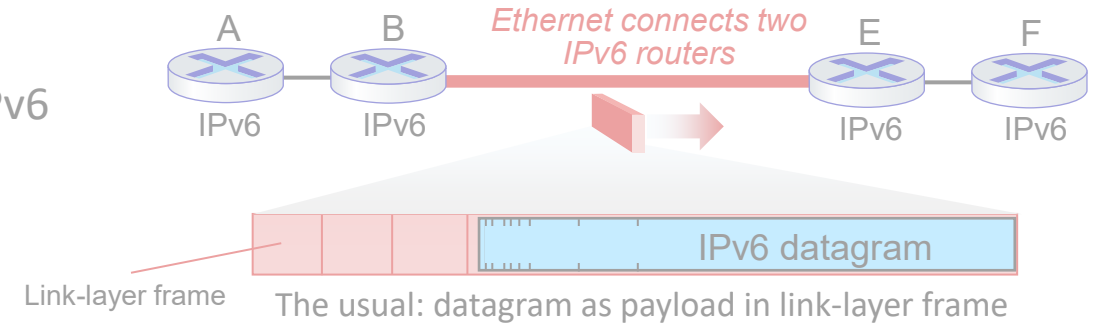



IPv4 network connecting two IPv6 routers

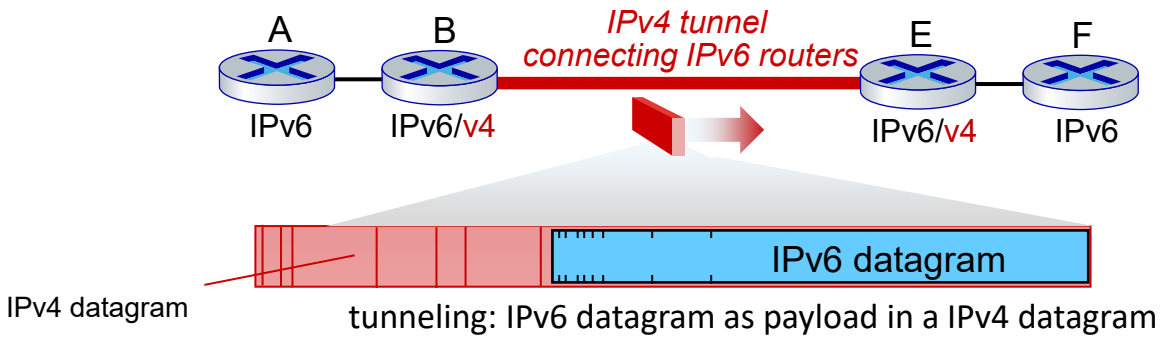


# Tunneling and encapsulation

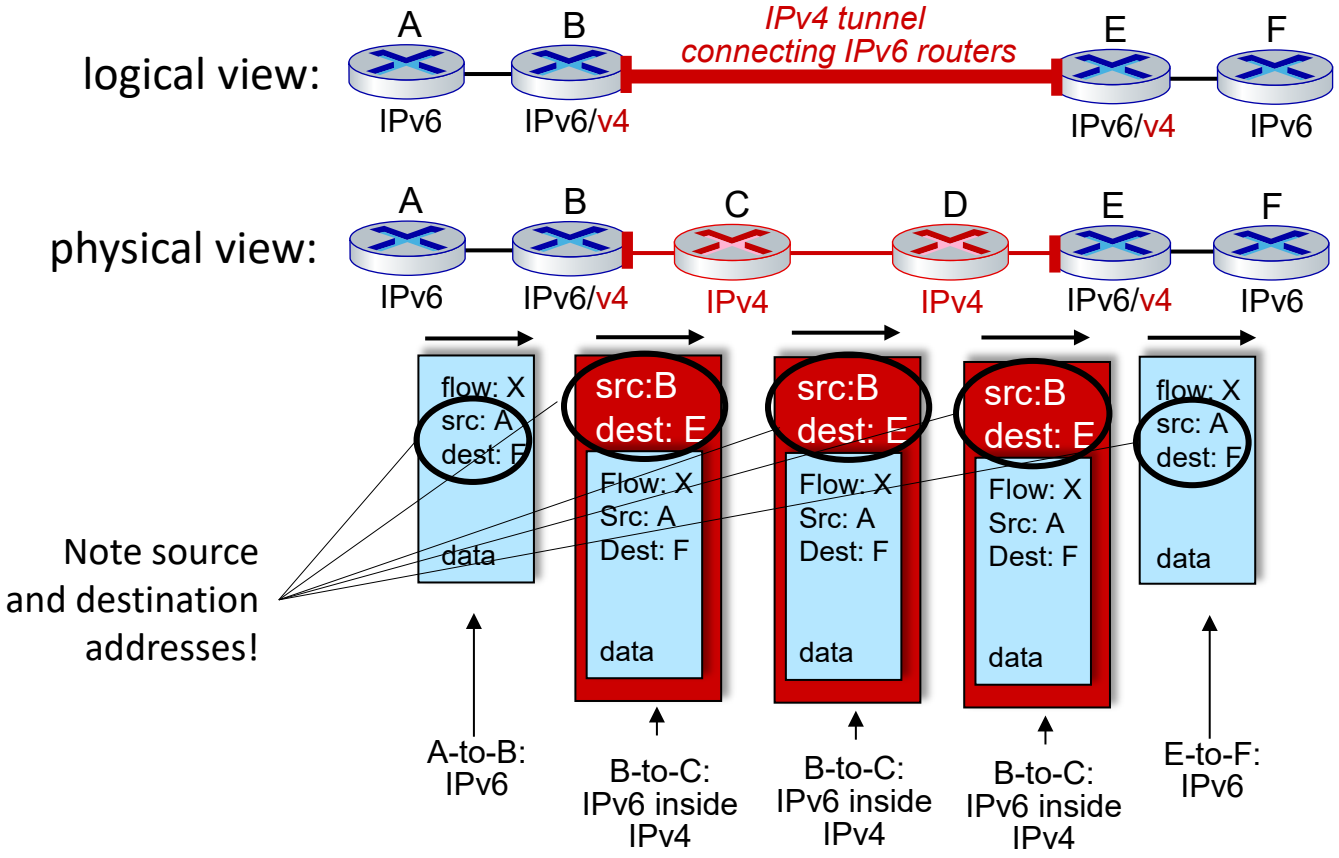
Ethernet connecting two IPv6 routers:




IPv4 tunnel connecting two IPv6 routers

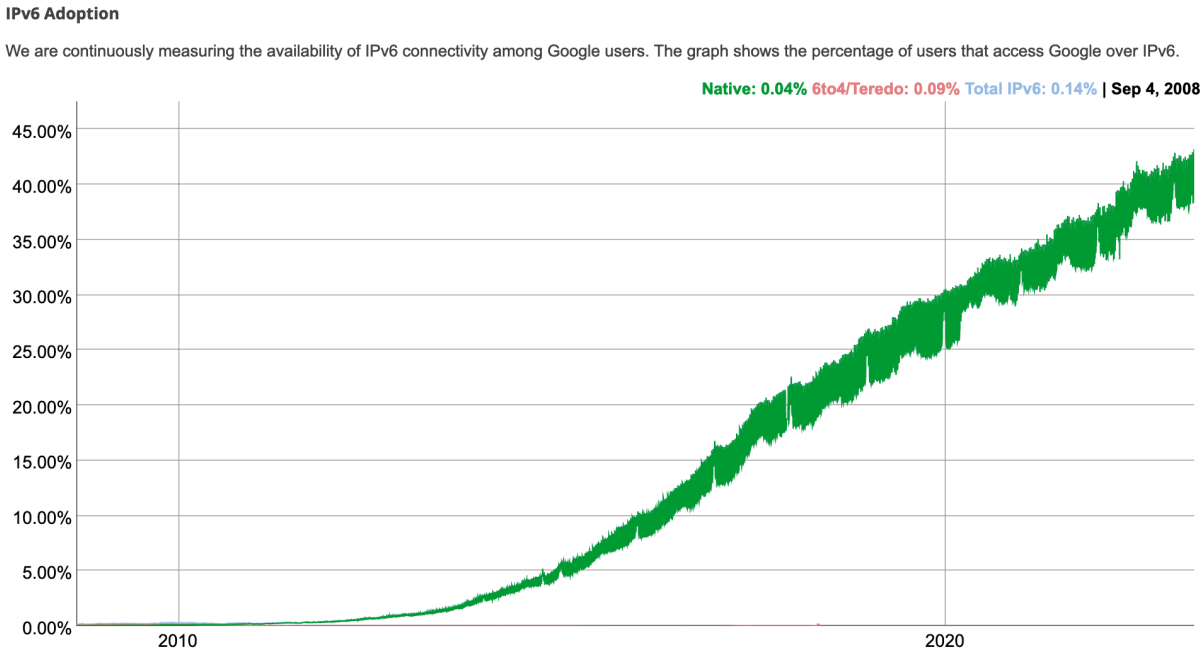


# Tunneling


logical view:
A
B
IPv4 tunnel
connecting IPv6 routers
E
F
IPv6
IPv6/v4
IPv6/v4
IPv6
physical view:
A
B
C
D
E
F
IPv6
IPv6/v4
IPv4
IPv4
IPv6/v4
IPv6
Note source
and destination
addresses!
flow: X
src: A
dest: F
data
src:B
dest: E
Flow: X
Src: A
Dest: F
data
src:B
dest: E
Flow: X
Src: A
Dest: F
data
src:B
dest: E
Flow: X
Src: A
Dest: F
data
flow: X
src: A
dest: F
data
A-to-B:
IPv6
B-to-C:
IPv6 inside
IPv4
B-to-C:
IPv6 inside
IPv4
B-to-C:
IPv6 inside
IPv4
E-to-F:
IPv6

# IPv6: adoption

- Google[1]: ~ 40% of clients access services via IPv6 (2023)
- NIST: 1/3 of all US government domains are IPv6 capable



# IPv6: adoption

- Google[1]: ~ 40% of clients access services via IPv6 (2023)
- NIST: 1/3 of all US government domains are IPv6 capable
- Long (long!) time for deployment, use
  - 25 years and counting!
  - think of application-level changes in last 25 years: WWW, social media, streaming media, gaming, telepresence, …
  - *Why?*

[1] https://www.google.com/intl/en/ipv6/statistics.html